

Generator-Based Framework for Automated Formal Verification of Parametrized RISC-V Cores

THESIS

Submitted for the degree of
Master of Science
in Embedded Systems Engineering

Department of Computer Science & Department of Microsystems Engineering
Faculty of Engineering
University of Freiburg



Submitted by:

Onur Altinordu

Supervisor: Anton Paule

First Examiner: Prof. Dr. Armin Biere

Second Examiner: Prof. Dr. Oliver Bringmann

March 23, 2026

Declaration

I hereby declare, that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used. Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I hereby also declare, that my thesis has not been prepared for another examination or assignment, either wholly or excerpts thereof.

Freiburg im Breisgau, March 23, 2026

Place, date

Signature

AI Tools Disclaimer

During the preparation of this thesis, artificial intelligence tools were used to assist with specific tasks as outlined in Table 1. The author has reviewed all AI-generated content and takes full responsibility for the accuracy, integrity, and originality of the thesis contents.

AI Tool	Purpose
ChatGPT 5.2	Extracting internal wire names from open-source RISC-V cores such as PicoRV32 for testing the framework
ChatGPT 5.2	Bug fixing for the source code of the verification framework
ChatGPT 5.2	Improving visuals in $\text{\LaTeX}$ document such as listings, tables, and diagrams
ChatGPT 5.2	Rephrasing author's writing to improve readability

Table 1: AI tools used during thesis preparation

Freiburg im Breisgau, March 23, 2026

Place, date

Signature

Abbreviations

CPU	Central Processing Unit 37
CSR	Control and Status Register 5
DUV	Design Under Verification 17
IR	Intermediate Representation 7
ISA	Instruction Set Architecture 1
LOC	Line of Code 46
LTL	Linear Temporal Logic 9
PC	Program Counter 4
RTL	Register-Transfer Level description 13
RVAVF	RISC-V Automatic Verification Framework 46
RVFI	RISC-V Formal Interface 6
RVFIC	RISC-V Formal Interface in Chisel 15
RVGM	RISC-V Golden Model 30
SVA	SystemVerilog Assertion 11

Abstract

Hardware generators can produce many processor variants from a single parameterized description, but verification remains tied to individual designs. Building formal verification environments by hand for each generated core does not scale, and the lack of strict semantics in existing formal interfaces prevents automation. This thesis investigates how a generator-based framework can enable automated and scalable formal verification of RISC-V processor implementations.

The proposed solution treats verification itself as a generator problem. Three cooperating generators share the same configuration that defines the hardware: a formal interface generator constructs a uniform observation layer over each core’s internal signals using SystemVerilog bind statements; a golden model generator produces a synthesizable reference that computes expected architectural effects; and a verification framework generator combines both, emits equivalence assertions, and produces the complete harness for bounded model checking. From one configuration, the system automatically generates all artifacts needed to verify that a core correctly implements its ISA.

Evaluation on four open-source RISC-V cores (PicoRV32¹, NERV², Ibex³, Hazard3⁴) demonstrates that the framework achieves proof coverage when formal interface signals follow consistent semantics. Where semantics diverge, the framework produces counterexamples that pinpoint signal mismatches rather than functional bugs. These findings motivate a signal contract that precisely defines the retirement-cycle requirements for each interface signal, establishing the foundation for portable, automated ISA verification across diverse RISC-V implementations.

¹<https://github.com/YosysHQ/picorv32>

²<https://github.com/YosysHQ/nerv>

³<https://github.com/lowRISC/ibex>

⁴<https://github.com/Wren6991/Hazard3>

Contents

Abbreviations

1	Introduction	1
2	Background	3
2.1	RISC-V Instruction Set Architecture	3
2.1.1	Instruction Formats	3
2.1.2	Extensions	4
2.2	RISC-V Formal Interface (RVFI)	6
2.3	Chisel and FIRRTL	7
2.4	Linear Temporal Logic (LTL)	9
2.5	SystemVerilog Assertions (SVA)	11
2.6	SystemVerilog Bind Construct	13
3	Related Work	16
3.1	Equivalence Checking Against a Reference Model	16
3.1.1	Simulation-Based Verification with Reference Models	17
3.1.2	RISC-V Formal	17
3.2	RVFC: RISC-V Formal in Chisel	18
4	Approach	19
4.1	Problem Definition	19
4.2	Methodology	20
4.3	RISC-V Formal Interface in Chisel (RVFIC)	23
4.3.1	Configuration Builder and Constants	27
4.3.2	Signal Mapping	28
4.3.3	Command-Line Interface	29
4.3.4	Map Specification Parser	30
4.3.5	Bind File Generation	31
4.3.6	Wrapper Generation	32
4.3.7	Generation Engine	34
4.3.8	End-to-End Signal Trace	34
4.3.9	Limitations and Comparison	36
4.4	RISC-V Golden Model (RVGM)	36
4.4.1	Configuration	39

4.4.2	Interface	40
4.4.3	Specification Modules	41
4.4.4	Golden Model	43
4.4.5	Specification Aggregator	44
4.4.6	Limitations and Comparison	45
4.5	RISC-V Automatic Verification Framework (RVAVF)	46
4.5.1	Configuration	47
4.5.2	Assertion Generation	49
4.5.3	Top-Level Harness	57
4.5.4	Formal Verification Flow	59
5	Evaluation	61
5.1	Experimental Setup	61
5.1.1	Target Cores	61
5.1.2	Verification Commands	62
5.1.3	State Space Reduction	62
5.2	Verification Results	63
5.3	Analysis of Results	65
5.3.1	Hazard3 RVFI Implementation	65
5.3.2	PicoRV32 Order Counter Timing	65
5.3.3	PicoRV32 Register File Initialization	65
5.3.4	Memory Interface Limitations	66
5.4	RVFIC Signal Contract	66
5.5	Discussion	68
6	Conclusion and Future Work	70
6.1	Conclusion	70
6.2	Future Work	71
A	Appendix	74
A.1	Source Code Repositories	74
A.2	PicoRV32 JSON Map	74
A.3	PicoRV32 Internal JSON Map	75

Introduction | 1

Modern processor design is shaped by rising complexity and dense integration of features. A single processor core may support multiple ISA extensions, configurable pipeline depths, optional floating-point or vector units, and various memory interfaces. Adding to this complexity is high variability. The same base architecture can yield dozens of distinct configurations tailored to different application domains.

This rising complexity has shifted the bottleneck in hardware development. Verification now dominates the effort. According to Foster, industry studies suggest that verification consumes up to 70% of the total design effort [1], and this proportion grows as designs become more configurable.

Hardware generators have emerged as a response to handle this variability and explore the design space. Using modern hardware construction languages, engineers describe processors as parameterized generators rather than fixed designs. A single generator can produce many concrete implementations by changing architectural parameters. This approach has made design reuse practical, but verification has not kept pace.

Ensuring that a processor correctly implements its ISA is a central verification challenge. The ISA defines a contract between hardware and software. It specifies how each instruction modifies registers, accesses memory, and updates the program counter. When hardware deviates from this contract, consequences range from subtle software bugs to complete system failure. Simulation-based verification tests a design against a finite set of input sequences, but cannot exhaustively cover all possible instruction sequences and processor states. Formal verification offers a complementary approach. It mathematically proves that a design satisfies its specification for all reachable states.

Applying formal verification to a processor requires constructing a verification environment. This includes a reference model capturing expected ISA behavior, an observation mechanism exposing architectural state changes, equivalence conditions, and a tool-specific harness. Such an environment is typically tailored to a single design and must be adapted whenever the processor changes. For generator-based designs producing many variants, assembling verification by hand does not scale.

A promising direction for closing this scalability gap is to extend the generator principle to formal verification itself. If verification environments could be generated alongside the hardware they verify, adapting to the same configuration parameters, formal methods might become practical for generator-based designs. A common formal interface could provide the foundation for such an approach, defining a uniform observation mechanism that abstracts away implementation details

while exposing architectural state changes. This raises the central question explored in this thesis: *How can a generator-based framework leveraging a common formal interface enable automated and scalable formal verification of RISC-V core implementations?*

Answering this question requires investigating several related concerns. What interface abstractions are needed to observe architectural behavior across diverse implementations? How can reference models and verification harnesses be generated from shared configurations? And what are the practical limits and tradeoffs of such an approach when applied to real processor designs? This thesis addresses these questions through the design, implementation, and evaluation of a verification framework targeting open-source RISC-V cores.

Background | 2

A processor implementation and its specification describe the same behavior at different levels of detail. A specification describes what a processor is required to do by defining the externally observable behavior of its instructions, such as how registers, memory, and the program counter are updated, without saying anything about internal structure. An implementation describes how this required behavior is realized in hardware, using design elements such as pipelines, control logic, and data paths, which may vary as long as the observable behavior matches the specification. Formal verification checks whether these two descriptions agree. This chapter covers the concepts needed to build such a verification system, including the target instruction set, the interface for observing instruction execution, the hardware generation tools used to construct verification components, and the formal techniques used to verify correctness.

2.1 RISC-V Instruction Set Architecture

RISC-V [2] is an open-standard RISC architecture developed at UC Berkeley. Unlike other ISAs such as ARM and x86, the RISC-V specification is freely available and extensible, which has led to a large ecosystem of open-source processor implementations. This openness is directly relevant to formal verification: because core designs and their source code are accessible, automated tools can analyze internal signals and compare execution behavior against an architectural reference model.

The ISA consists of a base integer instruction set and optional standard extensions. The two base variants, RV32I and RV64I, provide 31 general-purpose registers (x1–x31) and a hardwired-zero register (x0). RV64I extends RV32I with 64-bit registers and addresses, and adds word-sized operations (ADDW, SUBW, SLLW, etc.) that operate on 32-bit values and sign-extend results to 64 bits. All instructions are encoded in 32 bits, regardless of register width.

2.1.1 Instruction Formats

RISC-V defines six instruction formats: R, I, S, B, U, and J. All formats share a common 7-bit opcode field in bits [6:0]. Figure 2.1 shows the complete encoding structure.

Register-register operations (ADD, SUB, XOR) use R-type encoding with two source registers (rs1,

rs2), a destination register (rd), and two function fields (funct3, funct7) that distinguish operations sharing the same opcode.

Operations requiring an immediate value use I-type encoding, which allocates 12 bits for a sign-extended immediate. This format covers immediate arithmetic (ADDI, XORI) and load instructions (LW, LD).

Store instructions (SW, SD) use S-type, where the 12-bit immediate is split into two fields (imm[11:5] and imm[4:0]) to keep source register positions consistent with I-type.

Conditional branches (BEQ, BNE, BLT, BGE) use B-type with a 13-bit signed offset encoded in multiples of 2 bytes. Unconditional jumps (JAL) use J-type with a 21-bit signed offset. Both formats reorder immediate bits across the instruction word to maximize hardware sharing with other formats. Upper-immediate instructions (LUI, AUIPC) use U-type encoding, which places a 20-bit immediate directly in bits [31:12] of the result.

Because all base instructions are 32 bits wide, the PC must always be aligned to 4-bytes.

31	25	24	20	19	15	14	12	11	7	6	0		
funct7			rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]					rs1		funct3		rd		opcode		I-type
imm[11:5]			rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]			rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]								rd		opcode			U-type
imm[20 10:1 11 19:12]								rd		opcode			J-type

Figure 2.1: RISC-V instruction encoding formats. All instructions are 32 bits with a 7-bit opcode in bits [6:0].

2.1.2 Extensions

RISC-V defines standardized extensions that add extra functionality to the base ISA. Implementations select extensions based on their target domain while keeping them compatible with the base specification. This work targets three extensions: M (integer multiply/divide), C (compressed instructions), and Zicsr (CSR access). These three extensions are relevant because they are commonly implemented in the cores used as verification targets in this thesis, and because they display different aspects of the verification flow: M tests arithmetic corner cases such as division by zero, C introduces variable-length instruction encoding, and Zicsr adds stateful architectural registers that must be tracked across instructions.

M Extension

The M extension adds integer multiplication and division. All instructions use R-type encoding. MUL returns the lower XLEN bits of the product. MULH variants return the upper XLEN bits with different sign treatments. Division by zero and overflow (e.g., $-2^{XLEN-1} / -1$) have architecturally defined results, which makes them important verification targets. Table 2.1 lists the M extension instructions.

Table 2.1: M extension instructions.

Instruction	Operation
MUL	Multiply, returns lower XLEN bits
MULH	Multiply, returns upper XLEN bits (signed $\times$ signed)
MULHSU	Multiply, returns upper XLEN bits (signed $\times$ unsigned)
MULHU	Multiply, returns upper XLEN bits (unsigned $\times$ unsigned)
DIV	Signed division
DIVU	Unsigned division
REM	Signed remainder
REMU	Unsigned remainder

C Extension

The C extension provides 16-bit compressed encodings for common instructions. Compressed instructions expand to standard 32-bit equivalents during decode and do not extend architectural state. Instruction fetch must handle 16-bit alignment and mixed 16/32-bit streams. When a core has the C extension, the minimum instruction alignment relaxes from 4 bytes to 2 bytes, allowing the PC to point to any halfword-aligned address. Table 2.2 summarizes the C extension instruction categories.

Table 2.2: C extension instruction categories.

Category	Instructions
Integer compute	C.ADD, C.SUB, C.ADDI, C.LI, C.MV
Shifts	C.SLLI, C.SRLI, C.SRAI
Logical	C.AND, C.OR, C.XOR, C.ANDI
Loads	C.LW, C.LWSP, C.LD, C.LDSP (RV64)
Stores	C.SW, C.SWSP, C.SD, C.SDSP (RV64)
Control flow	C.J, C.JAL (RV32), C.JR, C.JALR, C.BEQZ, C.BNEZ

Zicsr Extension

The Zicsr extension provides atomic CSR access. CSRs encode processor state including privilege level, exception handling configuration, and performance counters. These instructions use I-type encoding with the CSR address in the immediate field. Table 2.3 lists the Zicsr instructions.

Table 2.3: Zicsr extension instructions.

Instruction	Operation
CSRW	Atomic read/write CSR
CSRWS	Atomic read and set bits in CSR
CSRWC	Atomic read and clear bits in CSR
CSRWRI	Atomic read/write CSR, immediate source
CSRWSI	Atomic read and set bits in CSR, immediate source
CSRWCi	Atomic read and clear bits in CSR, immediate source

2.2 RISC-V Formal Interface (RVFI)

RVFI is a signal-level interface that describes the architectural effects of each instruction retired by a RISC-V processor core. RVFI is not part of the official RISC-V specification; it was introduced by the *riscv-formal* project [3] as a community-driven convention for formal verification. Despite its unofficial status, RVFI has become widely adopted in both open-source and academic verification efforts because it provides a uniform observation point that is independent of a core’s internal microarchitecture.

RVFI models execution as a sequence of *retirement transactions*. Whenever an instruction retires, the core reports the instruction word together with the architecturally visible effects caused by that instruction. These effects include the program counter before and after retirement, any register file update, and any memory access. All RVFI signals are sampled synchronously and guarded by a validity signal (`rvfi_valid`) that indicates whether the interface carries a meaningful retirement transaction in the current cycle.

Table 2.4 lists the core RVFI signals used in this work. The interface separates control signals (valid, order, trap), instruction data, program counter values, register operands, and optional memory access fields. Signal widths depend on XLEN. Register data and addresses are XLEN bits wide, while the instruction word is always 32 bits and register address fields are always 5 bits.

Table 2.4: Selected RVFI signals (overview).

Signal	Width	Meaning
<code>rvfi_valid</code>	1	Indicates that the RVFI fields describe a retired instruction in this cycle.
<code>rvfi_order</code>	64	Retirement order counter (monotonically increasing for each retired instruction).
<code>rvfi_insn</code>	32	Retired instruction word (as executed).
<code>rvfi_pc_rdata</code>	XLEN	Program counter value before retirement (PC of the retired instruction).
<code>rvfi_pc_wdata</code>	XLEN	Program counter value after retirement (next architectural PC).
<code>rvfi_trap</code>	1	Indicates that the retired instruction caused a trap/exception.
<code>rvfi_halt</code>	1	Indicates that the core is halted (as defined by the RVFI specification).
<code>rvfi_intr</code>	1	Indicates that an interrupt is taken/observed at retirement.
<code>rvfi_rs1_addr</code>	5	Source register 1 index.
<code>rvfi_rs2_addr</code>	5	Source register 2 index.
<code>rvfi_rs1_rdata</code>	XLEN	Data read from source register 1.
<code>rvfi_rs2_rdata</code>	XLEN	Data read from source register 2.
<code>rvfi_rd_addr</code>	5	Destination register index.
<code>rvfi_rd_wdata</code>	XLEN	Data written to destination register.
<code>rvfi_mem_addr</code>	XLEN	Address used for the memory access of the retired instruction (if any).
<code>rvfi_mem_rdata</code>	XLEN	Data read from memory (for loads).
<code>rvfi_mem_wdata</code>	XLEN	Data written to memory (for stores).

Continued on next page

Table 2.4 – continued from previous page

Signal	Width	Meaning
<code>rvfi_mem_rmask</code>	XLEN/8	Byte-enable mask for memory reads.
<code>rvfi_mem_wmask</code>	XLEN/8	Byte-enable mask for memory writes.

The main purpose of RVFI is to enable architectural verification. A verification environment observes the sequence of retirement transactions produced by a core and compares each transaction against the output of an architectural reference model. If the core and the specification receive the same instruction stream and produce different architectural effects, the mismatch constitutes a verification failure.

Not every retired instruction updates all parts of the architectural state. An arithmetic instruction typically updates a destination register but does not access memory, while a store updates memory but does not write a register.

2.3 Chisel and FIRRTL

Chisel [4] is a hardware construction language embedded in Scala. Unlike traditional HDL’s such as Verilog and VHDL, which describe hardware directly as text that represents the structure and behavior of a circuit, Chisel uses Scala’s type system and abstraction to generate hardware. Designers write parameterized Scala classes that, when executed, produce a hardware graph. This approach enables generators rather than single fixed designs.

Chisel programs do not compile directly to Verilog. Instead, they emit FIRRTL, a compiler IR for digital circuits. The FIRRTL compiler applies a series of transformations such as type checking, width inference, optimization, and lowering, before emitting the final SystemVerilog output. This separation between the construction language and the output language allows the same Chisel source to target different backends (SystemVerilog, simulation models, or formal verification harnesses) without modifying the generator.

To illustrate the generator approach, consider a module that accumulates incoming data and optionally provides a saturating output port. Example 2.3.1 shows the Chisel source. The `width` parameter controls the data path width, and the boolean `hasSaturate` parameter determines whether the module includes an additional output that clips the accumulated value to its maximum.

Example 2.3.1: Parameterized accumulator generator in Chisel

```

1 class Accumulator(width: Int, hasSaturate: Boolean)
2   extends Module {
3     val io = IO(new Bundle {
4       val in    = Input(UInt(width.W))
5       val en    = Input(Bool())
6       val acc   = Output(UInt(width.W))
7       val sat   = if (hasSaturate)
8                     Some(Output(UInt(width.W)))
9                     else None
10    })
11    val max = (BigInt(1) << width) - 1
12    val reg = RegInit(0.U(width.W))
13    val sum = reg +& io.in // (width+1)-bit result
14    when(io.en) { reg := sum }
15    io.acc := reg
16    if (hasSaturate) {
17      val overflow = sum > max.U
18      io.sat.get := Mux(overflow, max.U(width.W), reg)
19    }
20 }

```

The two parameters are ordinary Scala values. Changing them does not require modifying the source: the generator produces structurally different modules depending on the configuration. Example 2.3.2 shows the SystemVerilog emitted for an 8-bit accumulator without saturation, and Example 2.3.3 shows the output for a 16-bit accumulator with the saturation port enabled.

Example 2.3.2: Generated SystemVerilog for Accumulator(8, false) (simplified)

```

1 module Accumulator(
2   input      clock,
3   input      reset,
4   input [7:0] io_in,
5   input      io_en,
6   output [7:0] io_acc
7 );
8   reg [7:0] reg_0;
9   always @(posedge clock) begin
10     if (reset)      reg_0 <= 8'h0;
11     else if (io_en) reg_0 <= reg_0 + io_in;
12   end
13   assign io_acc = reg_0;
14 endmodule

```


Example 2.3.3: Generated SystemVerilog for Accumulator(16, true) (simplified)

```

1 module Accumulator(
2   input      clock,
3   input      reset,
4   input [15:0] io_in,
5   input      io_en,
6   output [15:0] io_acc,
7   output [15:0] io_sat
8 );
9   reg [15:0] reg_0;
10  wire [16:0] sum = reg_0 + io_in;
11  wire      overflow = sum > 17'hFFFF;
12  always @(posedge clock) begin
13    if (reset)      reg_0 <= 16'h0;
14    else if (io_en) reg_0 <= sum[15:0];
15  end
16  assign io_acc = reg_0;
17  assign io_sat = overflow ? 16'hFFFF : reg_0;
18 endmodule

```

Several observations are worth noting. The Chisel source in Example 2.3.1 is a single parameterized class of 20 lines. From it, the FIRRTL compiler produces two structurally different SystemVerilog modules: the first has five I/O ports (Example 2.3.2, Lines 2:6) and no saturation logic, the second has six ports (Example 2.3.3, Lines 2:7) and includes the comparison and clipping assignment. The `io_sat` port does not exist in the first variant. It is not only tied to zero, it is absent from the port list entirely. Achieving this in plain Verilog would require preprocessor conditionals (``ifdef`) or `generate` blocks, both of which operate on text substitution and lack the type safety and composability of Scala’s control flow. This ability to produce structurally different hardware from a single source is the key advantage of generator-based design.

2.4 Linear Temporal Logic (LTL)

LTL, introduced by Pnueli [5], specifies temporal properties over executions modeled as infinite sequences of states. An execution is represented as a sequence $\sigma = s_0, s_1, s_2, \dots$, where each state assigns truth values to a fixed set of atomic propositions. LTL formulas are evaluated at positions of an execution and describe how properties relate to the ordering of states over time.

Formulas are constructed from atomic propositions using Boolean connectives and a small set of temporal operators. The operators **X** (next) and **U** (until) are taken as primitive, while **F** (eventually) and **G** (globally) are derived. The meaning of all operators is defined via a satisfaction relation $(\sigma, i) \models \varphi$, which specifies whether a formula φ holds at position i of an execution σ .

Definition 2.4.1 (**X** (next)). *The formula $\mathbf{X}\varphi$ holds at position i of an execution σ if and only*

if φ holds at the immediately following position $i + 1$, that is,

$$(\sigma, i) \models \mathbf{X}\varphi \iff (\sigma, i + 1) \models \varphi.$$

The operator $\mathbf{X}$ relates the current position of an execution to its direct successor and therefore captures one-step temporal progression. Figure 2.2 illustrates this by showing that satisfaction of φ is required exactly at the next state.

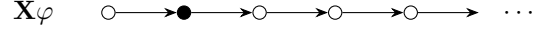


Figure 2.2: Directed-graph illustration of the $\mathbf{X}$ operator.

Definition 2.4.2 ($\mathbf{U}$ (until)). *The formula $\varphi_1 \mathbf{U} \varphi_2$ holds at position i of an execution σ if and only if there exists a position $j \geq i$ such that φ_2 holds at j and φ_1 holds at all positions from i to $j - 1$, that is,*

$$(\sigma, i) \models \varphi_1 \mathbf{U} \varphi_2 \iff \exists j \geq i. (\sigma, j) \models \varphi_2 \wedge \forall k. i \leq k < j \Rightarrow (\sigma, k) \models \varphi_1.$$

The operator $\mathbf{U}$ expresses a temporal dependency between two formulas: φ_1 must hold continuously until a position is reached where φ_2 holds. Figure 2.3 illustrates this behavior by marking a finite prefix of states satisfying φ_1 , followed by a state in which φ_2 becomes true.

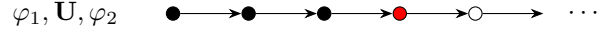


Figure 2.3: Directed-graph illustration of the $\mathbf{U}$ operator.

Definition 2.4.3 ($\mathbf{F}$ (eventually)). *The operator $\mathbf{F}$ is defined as an abbreviation in terms of the $\mathbf{U}$ operator:*

$$\mathbf{F}\varphi \equiv \text{true} \mathbf{U} \varphi.$$

Under this definition, $\mathbf{F}\varphi$ requires that there exists some future position of the execution at which φ holds. The illustration in Figure 2.4 emphasizes the existence of at least one future state satisfying φ , without constraining the states preceding it.

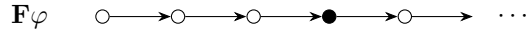


Figure 2.4: Directed-graph illustration of the $\mathbf{F}$ operator.

Definition 2.4.4 ($\mathbf{G}$ (globally)). *The operator $\mathbf{G}$ is defined as an abbreviation using $\mathbf{F}$:*

$$\mathbf{G}\varphi \equiv \neg \mathbf{F} \neg \varphi.$$

By definition, $\mathbf{G}\varphi$ requires that there is no future position at which $\neg\varphi$ holds; equivalently, φ must hold at all positions from the current one onward. Figure 2.5 illustrates this requirement by showing that every state along the execution satisfies φ .

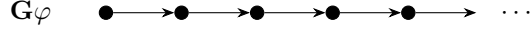


Figure 2.5: Directed-graph illustration of the $\mathbf{G}$ operator.

Application to hardware verification. As an example of how these operators compose, consider the property that every bus request must eventually receive an acknowledgment:

$$\mathbf{G}(req \rightarrow \mathbf{F}ack).$$

This formula states that at every point in the execution ($\mathbf{G}$), if a request is active, then there must exist a future state ($\mathbf{F}$) in which the acknowledgment eventually holds. Properties of this form are common in hardware verification and translate directly into SystemVerilog assertions, as shown in Section 2.5.

2.5 SystemVerilog Assertions (SVA)

SVA [6, 7] provide a declarative way to express expected design behavior over time. Assertions specify temporal properties over sequences of sampled design states and are evaluated with respect to a clocked execution model. Formal verification flows use SVA to cover, constrain, and verify design behavior.

SVA distinguishes between immediate assertions, which check a condition at a single point in time during procedural execution, and concurrent assertions, which describe how conditions must hold over multiple clock cycles. Concurrent assertions are the primary mechanism for formal verification and are therefore the focus here. They are written using **sequence** and **property** expressions and instantiated using assertion statements.

An **assert property** statement specifies a correctness requirement that the design must satisfy. In simulation, a violation produces an error. In formal verification, the tool proves that the property holds for all reachable states under the given assumptions, or it returns a counterexample.

Example 2.5.1: Assert property

```
1 property req_ack;
2   @(posedge clk) req |-> ##[1:3]ack;
3 endproperty
4 assert property (req_ack);
```

Example 2.5.1 specifies that whenever **req** is asserted, **ack** must follow within one to three clock cycles. A counterexample to this property indicates a functional violation.

An **assume property** statement constrains the environment in which the design operates. In formal verification, assumptions restrict the set of explored behaviors and are not checked as correctness requirements.

Example 2.5.2: Assume property

```

1 property stable_req;
2   @(posedge clk) req |-> ##1 req;
3 endproperty
4 assume property (stable_req);

```

Example 2.5.2 constrains the environment such that once `req` is asserted, it remains asserted for at least one additional cycle. Assumptions like this exclude unrealistic input sequences that could produce spurious counterexamples.

A `cover property` statement specifies a temporal pattern of interest and records whether that pattern can occur. Coverage properties measure which behaviors have been observed in simulation or are reachable in formal analysis.

Example 2.5.3: Cover property

```

1 property req_ack_seen;
2   @(posedge clk) req ##1 ack;
3 endproperty
4 cover property (req_ack_seen);

```

Example 2.5.3 records whether a request followed by an acknowledgment in the next cycle is reachable. Unlike assertions, coverage does not impose a correctness requirement.

The `restrict property` statement further limits the set of behaviors considered by a formal tool. Restrictions are similar in intent to assumptions but do not introduce proof obligations and are not evaluated for violations.

Example 2.5.4: Restrict property

```

1 property no_reset_after_init;
2   @(posedge clk) disable iff (reset) $rose(init_done) |-> !reset[*];
3 endproperty
4 restrict property (no_reset_after_init);

```

Example 2.5.4 excludes behaviors in which `reset` is asserted after initialization has completed, thereby eliminating executions that cannot occur in the intended operating environment.

Together, assertions, assumptions, coverage, and restrictions form the core of assertion-based verification in SystemVerilog. Assertions specify correctness requirements, assumptions and restrictions define the valid operating context, and coverage properties provide insight into reachable behaviors [6, 7].

Mapping LTL to SVA. The LTL property from Section 2.4 — $\mathbf{G}(req \rightarrow \mathbf{F} ack)$ — translates directly into a concurrent SVA property.

Example 2.5.5: LTL-style liveness assertion

```

1 property req_eventually_ack;
2   @(posedge clk) req |-> ##[0:$] ack;
3 endproperty
4 assert property (req_eventually_ack);

```

Example 2.5.5 encodes the same liveness requirement. Whenever `req` is asserted, `ack` must follow in the same or a later cycle. The `##[0:$]` delay range expresses the unbounded **F** operator. In bounded model checking, this unbounded range is evaluated only up to the bound k , so the tool can detect violations within the bound but cannot prove liveness for infinite sequences. It is important to note that the specification of requirements is not unique, since the same requirement can be expressed using different assertions and operators.

2.6 SystemVerilog Bind Construct

The SystemVerilog `bind` construct [8] is an elaboration-time mechanism that instantiates a module inside another module without modifying the RTL source code of the target module. The bound module has access to all signals in the target’s scope, but the target module is unaware of its presence: no ports are added, no logic is changed, and the synthesized netlist of the target remains identical with or without the `bind`. This non-invasiveness makes `bind` particularly suitable for verification, where assertion modules and observation logic must be attached to a design without altering its behavior.

An example assertion module is shown in Example 2.6.1. The module `mone_prop` contains only verification logic and therefore declares only input ports. The assertion checks the property that whenever signal `a` has a rising edge at a clock edge, signal `b` must hold in the next cycle.

Example 2.6.1: Assertion module

```

1 module mone_prop (input logic a, b, clk);
2   property P1;
3     @(posedge clk) $rose(a) | => b;
4   endproperty
5   A1: assert property (P1);
6 endmodule

```

The corresponding design module is shown in Example 2.6.2. It represents the RTL to be verified. The module `mone` provides the signals `req`, `ack`, and `clk`, which serve as inputs to the assertion module.

Example 2.6.2: Design module

```

1 module mone (input logic clk, req,
2               output logic ack
3               ...);
4   ...
5 endmodule

```

The testbench instantiates two instances of the design module, as shown in Example 2.6.3. The `bind` statement binds the assertion module `mone_prop` into the module type `mone` and creates an instance named `mp1`. Because the bind target is a module type rather than a specific instance, the binding applies to every instance of `mone` in the design hierarchy. Both `m1` and `m2` therefore receive an instance of the assertion module automatically.

Example 2.6.3: Testbench

```

1 module top;
2   ...
3   // instantiate design modules
4   mone m1 (clk, req1, ack1);
5   mone m2 (clk, req2, ack2);
6   bind mone mone_prop mp1 (req, ack, clk);
7   ...

```

The port list in the bind statement connects signals from the scope of the target module. In this example, `req`, `ack`, and `clk` are taken from inside `mone` and mapped to the inputs of `mone_prop` in the given order.

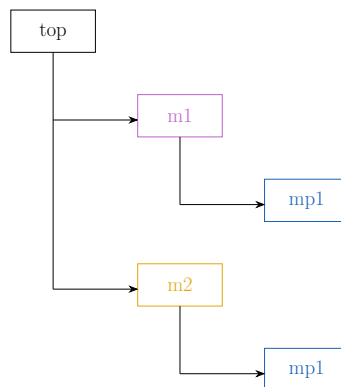


Figure 2.6: Hierarchy resulting from bind construct

The resulting hierarchy is illustrated in Figure 2.6. The assertion module instance `mp1` appears inside each design instance, producing `top.m1.mp1` and `top.m2.mp1`. This mechanism is central to

the RVFIC generator described in Chapter 4.3: RVFIC uses `bind` to attach a probe module to the core under verification, tapping internal signals without modifying the core's source code.

There are many ways of verifying a processor. Simulation was the dominant method for many years, but running enough test cases to cover all possible behaviors becomes impractical as designs grow. Formal methods address this by checking properties over all reachable states within a bounded depth. Different approaches have emerged over time: some compare the implementation against a reference model, others check each instruction in isolation, and others combine simulation with targeted formal checks. Each method has its own trade-offs in terms of effort, coverage, and the types of bugs it can find.

3.1 Equivalence Checking Against a Reference Model

A good approach to processor verification is to compare the implementation against an architectural reference model. Burch and Dill [9] introduced a technique that automatically verifies the control logic of pipelined processors by comparing each retired instruction’s effect against an unpipelined specification. Their method abstracts away the data path using uninterpreted functions, making the verification cost independent of data path width and register file size. This principle of comparing instruction-level behavior rather than internal microarchitectural state, remains the basis for most ISA-level verification approaches, including the framework presented in this thesis.

Gao and Melham [10] apply this principle to end-to-end formal verification of a CHERI-extended RISC-V processor. They use the Sail¹ ISA specification as the reference model and prove that the microarchitectural implementation produces the same architectural effects for every instruction, including complex capability-pointer operations. The work demonstrates that a golden ISA-level reference can guide the construction of assertions and proofs even for non-trivial architectural extensions.

Ploix et al. [11] extend this line of work to the CHERIoT-Ibex² processor, targeting *observational correctness*: the implementation is proven to match the Sail specification in terms of all externally observable behavior, including capability compression and security-relevant properties. This work demonstrates that golden-model-based formal verification can scale to a complete processor core.

¹<https://aldasair.github.io>

²<https://github.com/lowRISC/ibex>

Both of these works use the Sail language as their golden model. In contrast, the framework presented in this thesis uses a synthesizable golden model written in Chisel, which produces SystemVerilog that can be instantiated alongside the core under test in the same formal verification harness. This avoids the need for a separate co-simulation infrastructure and allows the comparison to be expressed directly as SystemVerilog assertions.

3.1.1 Simulation-Based Verification with Reference Models

Golden models are also used in dynamic verification flows that do not rely on formal proofs. Davidmann et al. [12] present a flow for RISC-V processors where constrained-random instruction streams are generated and executed both on the DUV and on a golden reference model. The verification environment compares architecturally visible outcomes at instruction retirement, detecting functional mismatches on a per-instruction basis. This approach can uncover subtle corner cases through extensive test generation, but it provides no completeness guarantee: only the executed test cases are checked.

Bruns et al. [13] combine golden-model checking with symbolic execution. Instead of generating concrete instruction streams, they explore instruction sequences symbolically, steering the search toward hard-to-reach paths and corner cases while still using the golden model as the correctness baseline. Compared to purely random testing, symbolic execution improves coverage systematically, but it remains bounded by the explored program length.

The framework in this thesis supports both formal and simulation-based verification. In simulation mode, waveform traces allow manual or automated comparison of the core’s RVFI output against the golden model. In formal mode, the same comparison is expressed as SystemVerilog assertions and checked exhaustively within a bounded model checking flow.

3.1.2 RISC-V Formal

An alternative to equivalence checking is to verify each instruction independently. The riscv-formal project [3] follows this approach. It defines a fixed set of RVFI signals that the core must expose, and provides a library of per-instruction checkers that compare the core’s reported retirement transaction against the expected architectural effect. Each checker is an SVA property that can be verified independently using bounded model checking.

The riscv-formal approach has two concepts that are relevant to this thesis. First, it relies on the core exposing RVFI signals with predetermined names and semantics. Cores that do not natively expose this interface require RTL modifications, which may be impractical for third-party IP or cores under active development. The RVFIC generator presented in this thesis addresses this limitation by automatically generating the RVFI interface from a signal map, without modifying the core’s source code.

Second, riscv-formal uses per-instruction checkers rather than a golden model. Each checker encodes the expected behavior of a single instruction as an SVA property. This is modular and allows adding instructions incrementally, but it means that the reference specification is distributed across many individual checkers rather than concentrated in a single executable model. The framework in this thesis takes the complementary approach: a single golden model implements the full ISA semantics, and the comparison is performed at the transaction level rather than per instruction.

3.2 RVFC: RISC-V Formal in Chisel

RVFC [14] is the most closely related work to the framework presented in this thesis. It proposes a verification methodology for RISC-V processors that is integrated into the Chisel hardware construction flow. Instead of relying on an external SystemVerilog-based verification environment, RVFC expresses both the design and the verification infrastructure in Scala/Chisel, aiming to make ISA-level formal checking more accessible for Chisel-based designs.

The RVFC approach follows a per-instruction verification pattern. The processor under verification exposes a formal interface that captures the architecturally visible effects of retired instructions. Modular checkers, one per instruction, compare the core’s reported behavior against the expected ISA semantics. This decomposition allows ISA compliance to be validated by local properties rather than through a monolithic reference implementation.

A central contribution of RVFC is its use of ChiselFV as a formal verification backend. This enables bounded model checking, k -induction, and property-directed reachability while keeping both the design and the checkers in the same language ecosystem. The authors demonstrate the methodology on a simple pipelined RISC-V core and report a branch-related bug caused by missing forwarding in the operand comparison, illustrating how ISA-level formal checking can expose microarchitectural corner cases.

Modern hardware construction languages allow processors to be described as parameterized generators rather than fixed RTL modules. A single generator can produce many concrete module instances by changing architectural parameters. Typical parameters include the register width, supported instruction set extensions and optional ISA features. Each parameter combination results in a distinct RTL design.

4.1 Problem Definition

From a verification perspective, this flexibility introduces a scalability challenge. There is no single processor to verify. Instead, verification must hold for a family of processors that differ in structure and functionality. Traditional verification flows are tailored to the DUV and therefore require manual updates whenever the design changes. This approach does not scale when designs are generated and many configurations are possible.

To reason about this design space, each processor instance in this work is described by a configuration vector $\theta \in \Theta$. The vector collects all architectural choices that influence the generated hardware. Changing θ results in a different core. The hardware generator maps this configuration to a synthesizable RTL description,

$$G_{\text{core}} : \Theta \rightarrow \mathcal{H}, \quad (4.1)$$

where $\mathcal{H}$ denotes the set of hardware implementations. For a specific configuration, the resulting design is

$$H_{\theta} = G_{\text{core}}(\theta). \quad (4.2)$$

Definition 4.1.1. *The verification objective is not tied to one design but must hold for all generated instances.*

$$\forall \theta \in \Theta : \text{Correct}(H_{\theta}).$$

Here, correctness means that the observable behavior of the hardware implementation H_{θ} conforms to its architectural specification for all valid instruction sequences and all reachable states.

Meeting this goal requires a verification approach that scales with the same parameter space as the hardware itself. Building and maintaining a separate testbench for every configuration would duplicate effort and quickly become unmanageable. Instead, verification must be constructed automatically from the same information that defines the hardware.

For this reason, the methodology in this work follows the same principle for both design and verification: if the hardware is generated, the verification environment should also be generated.

All generators in this work are implemented using Chisel and Scala. Chisel descriptions are lowered to FIRRTL and then emitted as SystemVerilog. The generated SystemVerilog is used for synthesis, simulation, or formal verification. JasperGold¹ is currently used as the formal backend.

4.2 Methodology

The proposed solution consists of three cooperating generators that together form a holistic formal verification environment. These components separate the concerns of specification, observation, and orchestration, and together provide an automated flow from configuration parameters to a formal proof. The three components are

- a golden reference model generator (gm),
- a formal interface generator (fi),
- a verification framework (vf).

All components are parameterized by the same configuration vector θ . For a given configuration, the generators produce matching instances,

$$gm_\theta = gm(\theta), \quad fi_\theta = fi(\theta), \quad vf_\theta = vf(\theta). \quad (4.3)$$

Using a shared configuration ensures that the DUV, the reference model, and the testbench are always consistent. When parameters change, all verification artifacts are regenerated automatically. As a result, the verification setup does not require manual updates when architectural features are enabled or disabled.

Conceptually, each component has a clear responsibility. The golden model describes the expected architectural behavior of the processor. The formal interface exposes architectural events from the RTL implementation in a standardized form. The verification framework connects these elements, adds the property set, and invokes the verification backend. This separation keeps the system modular and allows each part to evolve independently.

The golden model operates purely at the ISA level. It models the functional effect of instructions while ignoring timing and pipeline details. Instead of tracking internal microarchitectural signals, it maintains only architectural state. The exact structure of this state depends on the enabled ISA features. Besides the program counter, registers, and memory, it may also contain control and status registers, floating-point state, vector state, or other extension-specific data. For this reason, the state is written in an extensible form as

$$s_t = (pc_t, \mathbf{r}_t, \mathbf{m}_t, \dots, a_t^{(1)}, a_t^{(2)}, \dots), \quad (4.4)$$

¹Cadence JasperGold Formal Verification Platform, <https://www.cadence.com>

where pc_t is the program counter, $\mathbf{r}_t$ is the general-purpose register file, $\mathbf{m}_t$ is the memory state, and $a_t^{(1)}, a_t^{(2)}, \dots$ represent additional configuration-dependent architectural components such as control and status registers or extension-specific state. The ellipsis emphasizes that the state can grow with new extensions without changing the verification method.

Instruction execution updates the architectural state through a transition function,

$$s_{t+1} = f_\theta(s_t, i_t). \quad (4.5)$$

Here, s_t is the current architectural state, i_t is the instruction being executed at step t , f_θ is the configuration-dependent transition function that defines the ISA semantics, and s_{t+1} is the resulting state after execution. Repeated application of this transition produces an architectural execution trace,

$$\mathcal{T}_{\text{gm}} = (s_0, s_1, s_2, \dots), \quad (4.6)$$

which serves as the reference behavior during verification.

The RTL core, in contrast, exposes many internal signals that depend on the pipeline organization and implementation details. Directly comparing these signals to the golden model would tightly couple verification to the specific microarchitecture. To avoid this dependency, the formal interface translates cycle-level behavior into instruction-level events. For each retired instruction k , the wrapper emits a compact description of the architectural effect,

$$\tau_k = (pc_k, insn_k, s_k^{\text{pre}}, s_k^{\text{post}}). \quad (4.7)$$

This abstraction removes timing details and reduces verification to a comparison of architectural effects only. Both the RTL core and the golden model are mapped to the same event format, which enables direct and pipeline-agnostic checking of their behavior.

The verification framework orchestrates the complete process. It instantiates the wrapped RTL core and the wrapped golden model, binds the assertion set, and prepares the environment for the formal tool. All required files, including generated RTL, interface wrappers, properties, and tool configurations, are created automatically. The user interacts only with high-level parameters rather than editing tool scripts or file lists.

Figure 4.1 illustrates the structure of the complete system.

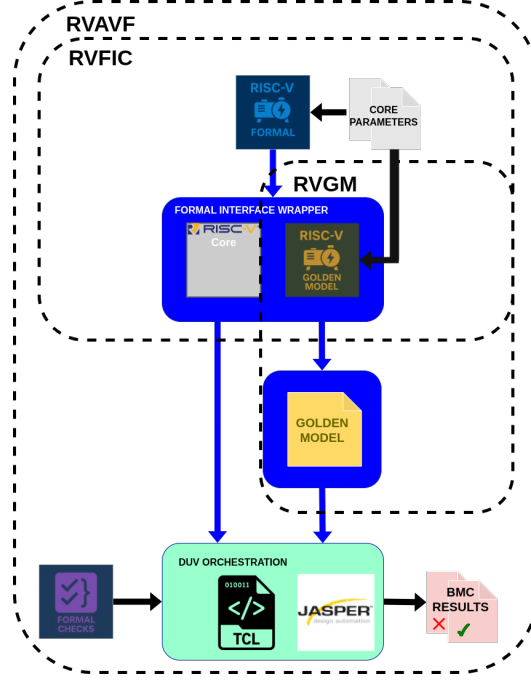


Figure 4.1: Generator-based verification flow. The verification framework instantiates the wrapped RTL core and the golden model and connects both to the generated assertion set.

All components are combined into a single verification instance.

Definition 4.2.1 (Verification Instance).

$$V_{\theta} = vf_{\theta}(f_{i_{\theta}}(H_{\theta}), f_{i_{\theta}}(gm_{\theta})).$$

Correctness is defined as equality between the architectural traces of both systems.

Definition 4.2.2 (Trace Equivalence).

$$\forall k : \tau_k^{core} = \tau_k^{gm}.$$

These relations are encoded as safety and liveness assertions. The framework supports both a default property set and additional user-defined assertions, allowing the verification to be extended with custom checks without modifying the infrastructure. Let P_{θ} denote the set of properties generated for configuration θ , which includes both the trace equivalence assertions and any user-defined checks.

The framework composes the complete testbench and invokes the verification backend,

$$vf_{\theta} = \text{Compose}(H_{\theta}, gm_{\theta}, f_{i_{\theta}}, P_{\theta}). \quad (4.8)$$

Definition 4.2.3 (Verification Condition).

$$V_\theta \models P_\theta.$$

If this condition holds, the verification instance satisfies all properties in P_θ , meaning the implementation is functionally equivalent to the golden model at the ISA level.

The build and execution flow is fully automated. Scala and Chisel generators are compiled using the Mill build system, while a Makefile coordinates generator execution, SystemVerilog emission, file collection, and invocation of the formal tool. This provides a single entry point for running verification and hides tool-specific details from the user.

Although the cores used in this work are generated using Chisel, the verification framework itself is not limited to generated designs. Any SystemVerilog RISC-V core that can be wrapped by the formal interface can be connected to the same golden model and verified using the same property set. Consequently, the framework supports both generator-based designs and handwritten RTL implementations with different parameters or ISA extensions.

In summary, the methodology treats verification itself as a generator problem. Given a configuration, the system automatically constructs a matching reference model, interface, and formal testbench. This enables consistent, reusable, and scalable ISA-level formal verification across a wide range of RISC-V core variants using one unified flow.

4.3 RISC-V Formal Interface in Chisel (RVFIC)

In this thesis, to make the framework connect a RISC-V core to an ISA-level formal verification flow, the formal interface generator RVFIC was developed. Given a RISC-V core and a JSON map file describing where architectural signals can be found inside the core, RVFIC produces two SystemVerilog files:

- A **bind file** containing a probe module and a SystemVerilog `bind` statement. The probe is inserted into the DUV hierarchy at elaboration time and taps the internal signals specified in the map.
- A **wrapper module** that instantiates the DUV, reads the probe outputs, and exports them through a uniform RVFI-style interface with consistent naming and widths.

Together, these files make the verification setup independent of the core’s internal signal names and structure. The verification framework interacts only with the wrapper’s ports and does not need to know how the core is organized internally.

Many RISC-V cores either do not expose RVFI signals, or they expose signals that are hard to reuse across different cores and configurations. Common problems are inconsistent signal names, missing fields, and signals that are only valid in specific pipeline stages. RVFIC solves this by handling two tasks separately. It first uses the map file to describe where each required signal can be found inside the core. It then generates the bind file and wrapper that expose these signals through one consistent interface.

Figure 4.2 illustrates the signal semantics with a concrete instruction sequence. The example executes three instructions: `addi x1, x0, 0x100` sets up a memory address, `sw x2, 0(x1)` stores the value in `x2` to that address, and `lw x3, 0(x1)` loads the stored value back into `x3`. Each signal

is meaningful only when `rvfic_valid` is high. The `rvfic_order` counter increments with each retirement, while memory signals show the store writing with `wmask=0xF` and the load reading with `rmask=0xF`.

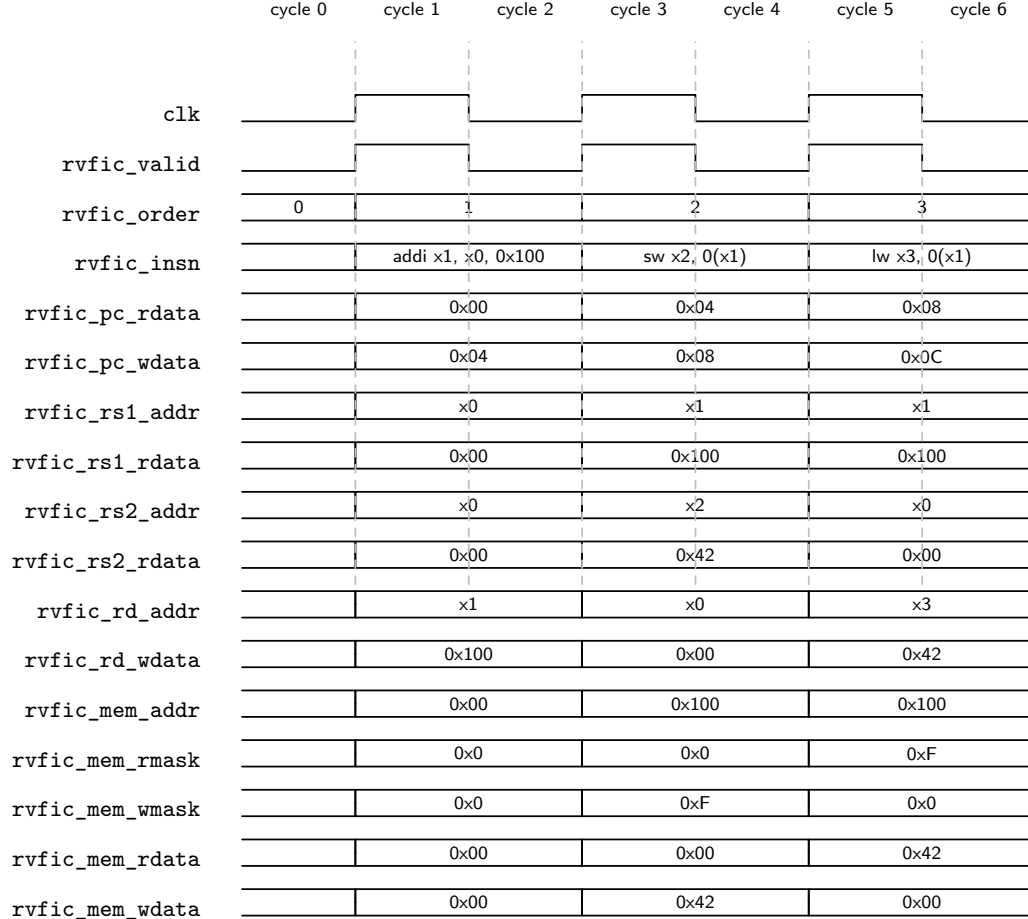


Figure 4.2: RVFIC waveform example with a load/store sequence.

RVFIC is designed for parameterized verification. When the user changes core parameters, the interface adapts automatically. RVFIC supports optional interface properties such as memory signals, CSR trace signals, and signals with different widths depending on XLEN. The verification environment sees one stable interface, even when the underlying core changes.

Figure 4.3 shows the simulation output of a PicoRV32 core wrapped with the base RVFI configuration: memory and CSR signals are disabled. The waveform contains only the core retirement signals: valid, order, instruction word, program counter, register addresses, and register data. This is the minimal interface that every RVFIC-wrapped core exposes.

RVFIC is also not limited to Chisel-generated cores. A core can be a Chisel generator or a SystemVerilog design. As long as RVFIC can access the required architectural signals (through ports

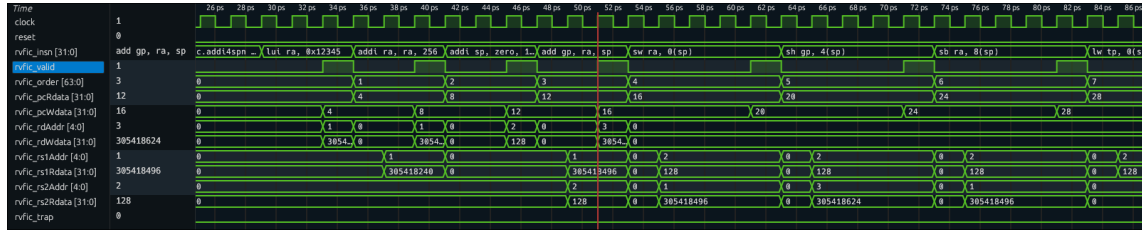


Figure 4.3: Simulation waveform of PicoRV32 wrapped with base RVFI signals (memory and CSR tracing disabled).

or through hierarchical paths in the design), it can wrap the core and export the same verification interface.

Figure 4.4 shows the RVFIC generation pipeline at a high level. The remainder of this section first explains the conceptual foundation of signal mapping, then describes the two generated outputs (bind file and wrapper), the engine that produces them, and finally the supporting components (map parser, configuration builder, and CLI).

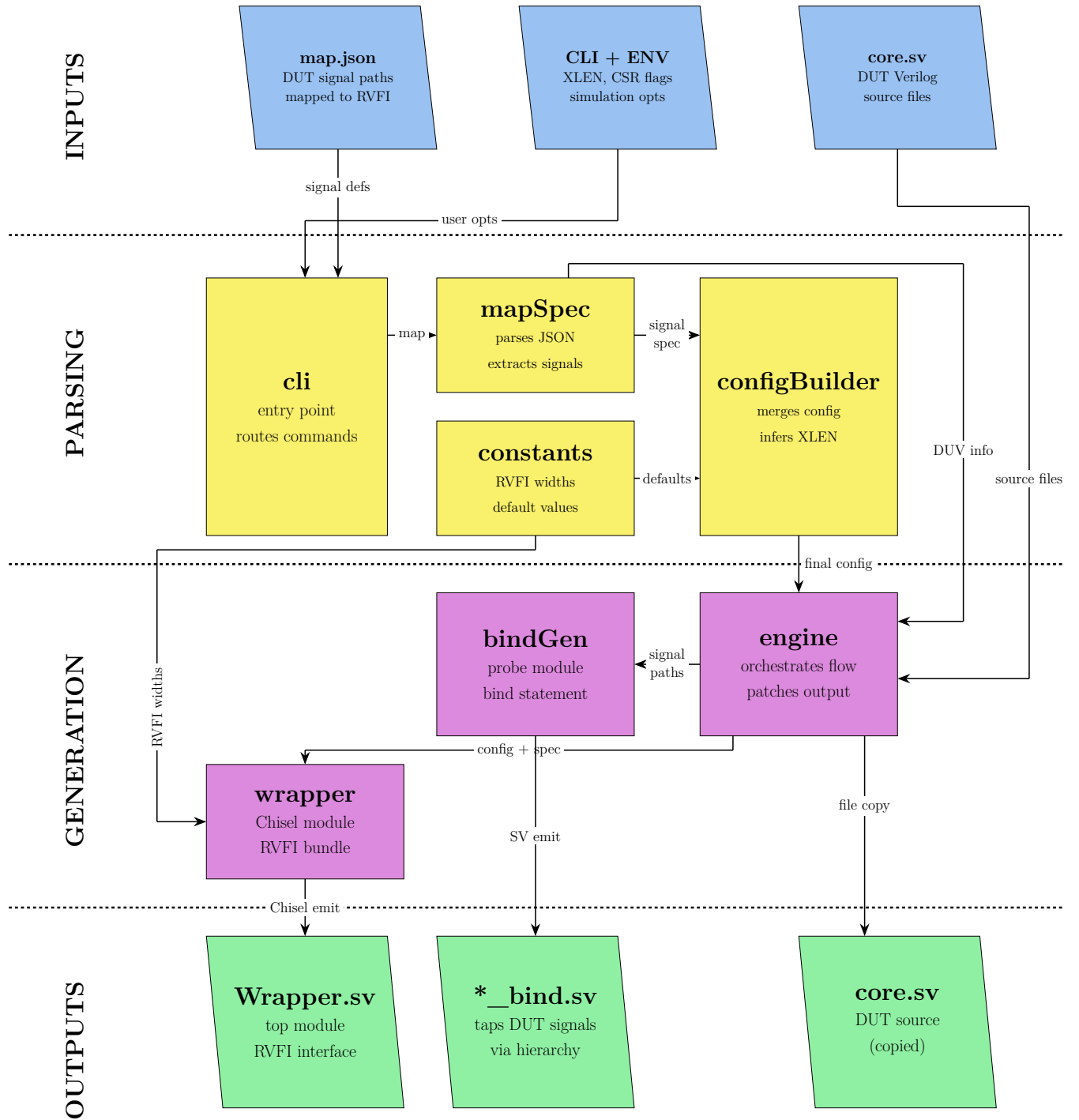


Figure 4.4: RVFIC generation pipeline, from inputs (map file, CLI options, DUV sources) through parsing and configuration to the final SystemVerilog outputs (bind file, wrapper module, and copied DUV sources).

4.3.1 Configuration Builder and Constants

The configuration builder (`configBuilder.scala`) produces the Scala case class named `RvficConfig` that all components of RVFIC use. This configuration determines which signals appear in the wrapper interface, their width, and which optional signal groups are enabled. RVFIC accepts configuration from the map file, CLI flags, and environment variables, with increasing priority. The resulting `RvficConfig` contains `xlen` (32 or 64), `orderBits` (retirement counter width), `nret` (fixed at 1), `enableMemorySignals`, and a nested `RvficCsrConfig` with per-register trace flags.

The builder takes initial values from the map specification where possible. For example, `XLEN` is taken from the width of the `rvfic_pc_rdata` signal. If the map declares it as 32 bits wide, the builder assumes RV32; if 64, RV64. Similarly, the order counter width is taken from the `rvfic_order` entry. These inferred values serve as defaults that can be overridden.

The builder also controls optional interface groups. RVFIC can export memory signals (`rvfic_mem_addr`, `rvfic_mem_rdata`, etc.) and CSR trace signals (`rvfic_csr_mcycle_rdata`, etc.), but only when the user has requested the group, and the map contains the required signal entries. If the map lists signals whose names start with `rvfic_csr_`, the builder automatically detects which CSR registers are available and enables tracing for them. This detection can be refined through CSR profiles (`none`, `minimal`, `machine_full`, `all`) or per-register flags.

Figure 4.5 shows the simulation output of an Ibex core wrapped with both memory signals and CSR tracing enabled. Compared to the PicoRV32 waveforms in Figures 4.3 and 4.6, the interface now includes CSR trace ports for `mcycle`, `minstret`, and `mstatus`, each with four sub-signals (`rdata`, `wdata`, `rmask`, `wmask`). The Ibex core supports these machine-mode CSRs natively, so the builder detected them from the map and enabled the corresponding trace groups. This demonstrates how the same generator adapts to a different core with different capabilities. PicoRV32 maps do not include CSR entries, so no CSR ports appear; the Ibex map does, and the interface expands accordingly.

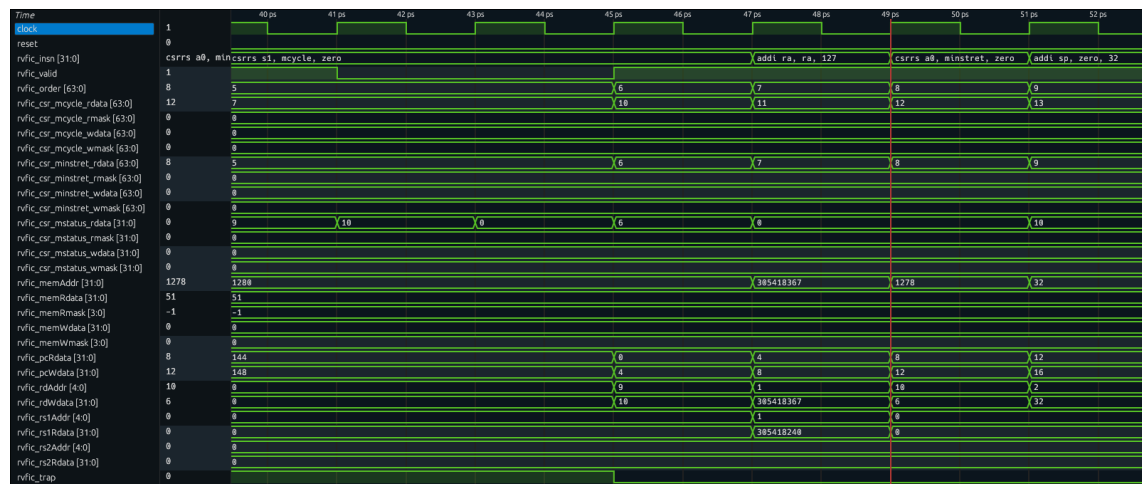


Figure 4.5: Simulation waveform of Ibex with memory and CSR tracing enabled. CSR trace ports for `mcycle`, `minstret`, and `mstatus` appear alongside the memory and base retirement signals.

All RVFIC components share a set of fixed constants defined in `RvficConstants` (`constants.scala`). Table 4.1 lists the key values. Most RVFI signal widths (register data, addresses, and the program

counter) are derived from XLEN. A few widths are fixed regardless of XLEN. The instruction word is always 32 bits because the base RISC-V encoding is 32 bits wide. Register address fields are always 5 bits. Memory masks are derived as $XLEN/8$, giving 4 bits for RV32 and 8 bits for RV64.

Table 4.1: Key constants in `RvficConstants`.

Constant	Value	Description
<code>instructionWidth</code>	32	Instruction word width (fixed)
<code>registerAddrWidth</code>	5	Register address width (fixed)
<code>defaultXlen</code>	32	Default register width
<code>minXlen, maxXlen</code>	32, 64	Supported XLEN range
<code>defaultOrderBits</code>	64	Retirement order counter width
<code>minOrderBits, maxOrderBits</code>	1, 128	Valid order width range
<code>memoryMaskWidth32</code>	4	Memory mask for RV32 (32/8)
<code>memoryMaskWidth64</code>	8	Memory mask for RV64 (64/8)
<code>maxPortWidth</code>	128	Maximum allowed signal width

4.3.2 Signal Mapping

Connecting a verification framework to an arbitrary RISC-V core can be a hard task to achieve. Cores differ not only in their microarchitecture but also in how they expose architectural state. Some cores provide RVFI ports, while others bury retirement signals deep within their pipeline hierarchy. Signal naming conventions vary widely, and even cores implementing identical ISA subsets may use completely different internal structures. A verification framework that hardcodes these details becomes brittle, since each new core requires modifications to the framework itself.

RVFIC addresses this challenge through a declarative mapping specification. The map file serves as an abstraction layer between core-specific implementation details and the RVFI interface that the verification flow expects. The specification uses JSON as its format for two practical reasons: JSON parsers exist in virtually every programming language including Scala and JSON’s simplicity reduces the likelihood of syntax errors compared to more expressive formats.

The map specification deliberately separates *what* signals to expose from *how* to access them. A mapped signal can originate from two distinct sources:

- **Port signals** exist as explicit top-level ports of the DUV module. RVFIC treats these as part of the blackbox interface and generates appropriate Chisel IO declarations.
- **Bind signals** exist inside the DUV hierarchy and must be accessed using a generated SystemVerilog `bind` probe. This mechanism allows tapping internal wires without modifying the DUV source.

This distinction determines how RVFIC generates the wrapper. Port signals appear in the blackbox module declaration, while bind signals require probe modules and `bind` statements that reach into the design hierarchy. The map can also specify expressions, which are pieces of combinational logic that derive RVFI values from multiple DUV signals. For example, a core might not have a single “instruction retired” signal, but the map can express this as a combination of other internal signals:

Example 4.3.1: Expression-based signal mapping

```

1 "rvfic_valid": {
2   "path": "resetn && dbg_valid_insn && launch_next_insn",
3   "width": 1
4 }

```

Rather than requiring users to modify the DUV or write custom wrapper logic, the map specification reduces this complexity.

The core developer, who understands the internal structure of their design, provides the map file alongside the RTL. This parallels how core developers already provide testbenches, synthesis constraints, and documentation. The map file becomes part of the core’s deliverables, enabling verification teams to use RVFIC without needing to reverse-engineer the core’s internals. Core developers know which internal signals correspond to architectural state, which pipeline stage produces valid retirement information, and which signals require combinational derivation. Encoding this knowledge once in a map file makes it reusable across all verification efforts for that core.

An alternative future direction would be automatic signal discovery through machine learning. A tool that could analyze RTL and find RVFI-equivalent signals would eliminate the need for manual map creation entirely. However, modern RISC-V cores are architecturally complex. Deep pipelines, speculative execution, and complex microarchitectural state make automatic identification of retirement-consistent signals a challenging research problem. The map specification provides a practical solution for today while remaining compatible with future automation.

In practice, the examples in this thesis primarily use cores that already expose RVFI signals, such as PicoRV32. This choice is due to a practical constraint: demonstrating RVFIC’s capabilities requires cores with known-correct signal mappings. Using pre-existing RVFI signals provides ground truth for validating that RVFIC’s generated wrappers function correctly.

This does not lower RVFIC’s value. The framework’s contribution is in its ability to adapt to arbitrary signal locations and naming conventions, not in requiring cores to lack RVFI support. A core with native RVFI signals at non-standard paths, with different widths, or requiring parameter-dependent signal selection still benefits from RVFIC’s mapping abstraction. The PicoRV32 internal map (Appendix A.3) demonstrates this by tapping signals through hierarchical paths and expressions rather than using the core’s RVFI ports directly.

4.3.3 Command-Line Interface

The CLI (`cli.scala`) is the entry point of RVFIC. It collects user inputs, validates them, builds a final configuration, and calls the generation engine. The CLI does not generate hardware by itself. It is usually called through a Makefile target that builds the full Mill command and passes all required paths and parameters, but it can also be invoked directly.

The primary use case of the cli is to wrap a SystemVerilog DUV using the signal locations described in a JSON map file. A typical invocation looks like this:

Example 4.3.2: RVFIC CLI invocation

```

1 ./mill -i rvfic.runMain rvfic.core.RvficCli wrapFromMap \
2   --map path/to/map.json \
3   --sv path/to/core.sv \
4   --xlen 32 \
5   --enableCsr true --csrProfile minimal

```

The two required arguments are the map file (`-map`) and one or more SystemVerilog source files (`-sv`). Optional flags control how the interface is generated:

- `-xlen`: register width (32 or 64),
- `-enableMemorySignals`: toggle the memory signal group,
- `-enableCsr`, `-csrProfile`, `-traceCsr`, `-disableCsr`: control CSR tracing granularity,
- `-gmodel`: switch to golden model naming mode, adding a `spec` tag to signal names to avoid collisions when both DUV and reference model are instantiated together,
- `-vcd`: when provided, RVFIC runs a short Verilator² simulation after generation and writes an FST waveform for debugging.

Without `-vcd`, the command only generates wrapper and bind files. With `-vcd`, it additionally invokes the simulation orchestrator to produce a waveform trace.

RVFIC accepts configuration from the map file's `rvficConfig` section, CLI flags, and environment variables. Map file settings have the lowest priority and are overridden by CLI flags, which are in turn overridden by environment variables. This layering allows quick experiments by setting a single environment variable without editing map files or build scripts. For example, a user can run `RVFIC_XLEN=64 make rvfic_wrap ...` to temporarily switch from RV32 to RV64 without touching the map.

The second use case is to wrap the RVGM golden model. In this case, external SystemVerilog sources are not used because the golden model is generated internally as a Chisel module. Instead, it accepts flags for configuring RVGM itself (`-gmXlen`, `-gmEnableM`, `-gmEnableC`, etc.) alongside the usual RVFIC interface flags. Both commands share the same configuration merging logic and produce a wrapper module and a bind file, written to the output directory.

4.3.4 Map Specification Parser

The Map Specification Parser (`mapSpec.scala`) implements the JSON parsing for RVFIC's signal mapping system. It transforms a JSON map file into Scala data structures that other RVFIC components use.

The map specification describes the structure of the mapping in a clear and explicit way. Each signal is represented as a separate entry that states its name, direction, bit width, and how it is accessed, as well as its location in the design if needed. Based on this information, signals can be distinguished by how they are connected, and the required connection paths can be determined

²<https://verilator.org>

directly. Reset behavior is described explicitly by naming the reset signal and its polarity, so reset handling is clear. All of this information is brought together in a single description that defines the design being verified, its clock and reset signals, and the complete set of interface connections, with optional settings for memory support, design parameters, and additional configuration.

DUV parameters receive type-aware parsing. Numeric values are converted to strings while preserving integer representation, and string values pass through unchanged. This allows SystemVerilog literals like "32'h00000020" or identifiers like "ibex_pkg::RV32MNone". For example:

Example 4.3.3: DUV parameter overrides

```
1 "parameters": { "ENABLE_COUNTERS": 1, "PROGADDR_IRQ": "32'h00000020", "RV32M":
2 "ibex_pkg::RV32MNone" }
```

The optional `rvficConfig` section allows the map to specify RVFIC generation settings such as register width, retirement order counter width, memory interface toggle, and per-CSR trace enables. These overrides take precedence over defaults taken from the DUV or command-line arguments.

The Scala Map data structure derived from the JSON Map serves as the single source of truth for the other components of RVFIC. The probe generation uses it to create the required observation logic, the wrapper generation uses it to instantiate and connect the DUV, and the configuration builder merges any overrides with other configuration sources. Appendix A.2 and Appendix A.3 provide complete map files for the PicoRV32³ core and demonstrate both supported formats (internal wire map and external port map).

4.3.5 Bind File Generation

The Bind Generator (`bindGen.scala`), is responsible of generating a SystemVerilog file that gives RVFIC access to signals inside the DUV without modifying its source code. The generated file contains two parts: a probe module and a `bind` statement that inserts the probe into the DUV hierarchy.

The probe module declares an input for each DUV signal that RVFIC needs to observe and an output for each RVFI signal the wrapper must export. The `bind` statement then instantiates this probe inside the DUV module, connecting each input to the corresponding hierarchical signal path from the map. The following abbreviated example shows the structure for PicoRV32:

³<https://github.com/YosysHQ/picorv32>

Example 4.3.4: Generated bind probe for PicoRV32

```

1 module picorv32_RvficProbe(
2   input      clk,
3   input      reset,
4   input [31:0] sig_rvfic_insn,    // DUV signal in
5   input      sig_rvfic_valid,
6   output [31:0] probe_rvfic_insn, // probe output
7   output      probe_rvfic_valid,
8   // ... remaining signals
9 );
10 assign probe_rvfic_insn = sig_rvfic_insn;
11 assign probe_rvfic_valid = sig_rvfic_valid;
12 endmodule
13 bind picorv32 picorv32_RvficProbe rvfic_probe_inst (
14   .clk(clk),
15   .reset(!resetn),
16   .sig_rvfic_insn(rvfi_insn),    // map path
17   .probe_rvfic_insn(),
18   .sig_rvfic_valid(rvfi_valid),
19   .probe_rvfic_valid()
20 );

```

Only signals marked as bind type in the map appear in the probe. Port signals (Section 4.3.2) are handled directly by the wrapper’s blackbox interface and do not need probe entries.

The bind generator supports three mapping forms beyond direct forwarding:

- **Counter mappings** synthesize a signal inside the probe. The most common example is the instruction order field. When a core does not provide a monotonically increasing order value, RVFIC implements it as a register in the probe that increments on each retirement event.
- **Expression mappings** derive one output from several DUV signals using the expression string from the map. The generator extracts the referenced signal names, creates probe inputs for each, and assigns the output using the translated expression. For instance, the PicoRV32 internal map expresses the valid signal as `resetn && dbg_valid_insn && launch_next_insn`.
- **Semantic overrides** add small pieces of sequential logic to shape signals for reliable formal checking. For example, PicoRV32’s memory read data is only valid on the bus during the cycle it arrives. The probe holds the last value in a register and multiplexes between the live bus data and the held value, ensuring the wrapper always sees consistent data at retirement time.

Bind Generator takes the parsed map specification and returns the complete SystemVerilog source as a string. The engine (Section 4.3.7) writes this string to the output directory alongside the wrapper and the DUV sources.

4.3.6 Wrapper Generation

The wrapper (`wrapper.scala`) is the top-level Chisel module that the verification framework interacts with. It instantiates the DUV, exports the RVFIC IO, and lets the bind probe (Section 4.3.5) fill in

the signal values after elaboration.

The file defines two wrapper classes. One handles SystemVerilog cores by instantiating the DUV as a blackbox, whose ports are built dynamically from the map file. The other handles the golden model. Instead of treating the golden model as a blackbox, the wrapper directly instantiates the Chisel `GoldenModel` generator and, during elaboration, resolves the map's signal paths against the generator's IO bundle to connect the RVFIC outputs.

Clock and reset deserve explicit treatment because they differ across cores. The map specifies the clock name, the reset name, and whether reset is active-low. The wrapper connects its own clock to the DUV clock port and applies polarity inversion when required:

Example 4.3.5: Reset polarity handling in the wrapper

```
1 rs := (if (spec.reset.activeLow) ~reset.asBool else reset.asBool)
```

RVFIC IO bundle is parameterized by `RvficConfig`. It always contains the base retirement signals (valid, order, trap, instruction, PC, register addresses and data). Memory signals (`memAddr`, `memRmask`, etc.) only exist when `enableMemorySignals` is true. CSR trace ports are generated dynamically, and contains four ports (rmask, wmask, rdata, wdata) for each enabled CSR register. This means the wrapper's IO shape adapts automatically to the configuration without any manual changes.

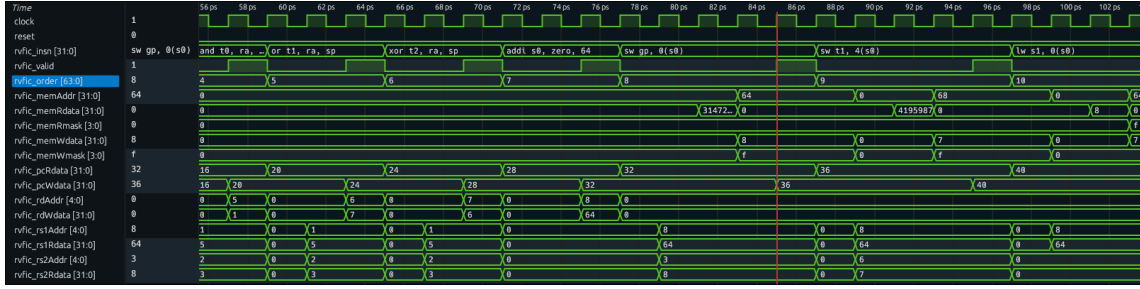


Figure 4.6: Simulation waveform of PicoRV32 with memory signals enabled. The memory group (`memAddr`, `memRdata`, `memWdata`, `memRmask`, `memWmask`) is now visible in the interface.

Figure 4.6 shows the effect of enabling memory signals on the same PicoRV32 core. Compared to Figure 4.3, the waveform now includes `rvfic_memAddr`, `rvfic_memRdata`, `rvfic_memWdata`, `rvfic_memRmask`, and `rvfic_memWmask`. These signals appear because the configuration set `enableMemorySignals` to true, causing the RVFIC IO bundle to include the memory fields. No other part of the wrapper or bind generator changed and only the configuration parameter differs.

When the wrapper is generated for the golden model side, RVFIC uses a naming mode that adds a `spec` tag to all signal names. This allows a wrapped DUV and a wrapped golden model to coexist in the same top-level formal environment without port name collisions.

4.3.7 Generation Engine

The engine (`engine.scala`) orchestrates the full wrapping process. It takes the parsed map specification, a list of SystemVerilog sources, and an output directory, then executes four steps in order:

1. Build the final `RvficConfig` via the configuration builder.
2. Copy the DUV's SystemVerilog sources into the output directory so that all generated artifacts and the original RTL are self-contained in one place.
3. Generate the bind file using the `Bind Generator`, which produces the probe module and the `bind` statement.
4. Generate the wrapper via Chisel elaboration and patch it to connect probe signals.

In the patching step in step 4, Chisel emits the wrapper as a generic SystemVerilog module. At elaboration time, the wrapper cannot reference internal DUV signals because those signals only become accessible after the bind probe is inserted into the DUV hierarchy. The emitted wrapper therefore contains zero-valued placeholder assignments for each RVFIC port. The engine replaces these placeholders with assignments that read from the bind probe instance. For example, a placeholder like

Example 4.3.6: Placeholder assignment before patching

```
1 assign rvfic_valid = 1'h0;
```

becomes

Example 4.3.7: Patched probe connection

```
1 assign rvfic_valid = core.rvfic_probe_inst.probe_rvfic_valid;
```

This two-stage approach keeps the Chisel wrapper independent of core-specific hierarchy paths. All core-specific knowledge stays in the map and in the generated bind file; the engine only coordinates how the pieces are produced and connected.

If the map includes DUV parameter overrides, the engine also patches the DUV instantiation to add a SystemVerilog parameter block. This allows RVFIC to configure the core (for example, enabling `RISCV_FORMAL` or disabling interrupts) without modifying the original source files.

4.3.8 End-to-End Signal Trace

To illustrate how the components described above work together, this subsection traces a single signal from its JSON map entry through each RVFIC stage to the final SystemVerilog output. The signal chosen is `rvfic_valid`, which indicates that the core has retired an instruction.

The PicoRV32 internal map defines the valid signal as an expression over three internal wires:

Example 4.3.8: Map entry for rvfic_valid

```

1 "rvfic_valid": {
2   "path": "resetsn && dbg_valid_insn && launch_next_insn",
3   "width": 1
4 }

```

Because `path` contains operators rather than a single hierarchical name, the map parser (Section 4.3.4) classifies this entry as an expression mapping and extracts the three referenced signal names.

Bind generation. The bind generator (Section 4.3.5) creates a probe input for each referenced signal and an output for the RVFI value:

Example 4.3.9: Probe module ports and assignment for rvfic_valid

```

1 // inside the probe module
2 input      sig_resetsn,
3 input      sig_dbg_valid_insn,
4 input      sig_launch_next_insn,
5 output     probe_rvfic_valid,
6 // ...
7 assign probe_rvfic_valid =
8   sig_resetsn & sig_dbg_valid_insn & sig_launch_next_insn;

```

The `bind` statement then connects each `sig_*` input to the corresponding DUV-internal wire:

Example 4.3.10: Bind statement connecting probe to DUV signals

```

1 bind picorv32 picorv32_RvficProbe rvfic_probe_inst (
2   .sig_resetsn(resetsn),
3   .sig_dbg_valid_insn(dbg_valid_insn),
4   .sig_launch_next_insn(launch_next_insn),
5   .probe_rvfic_valid(),
6   // ...
7 );

```

The Chisel wrapper (Section 4.3.6) declares `rvfic_valid` as a 1-bit output in the `RvficBundle` and initializes it to zero during elaboration:

Example 4.3.11: Chisel-emitted placeholder assignment

```

1 output      rvfic_valid,
2 // emitted by Chisel:
3 assign rvfic_valid = 1'h0;

```

The engine (Section 4.3.7) replaces the placeholder assignment with a connection to the probe instance:

Example 4.3.12: Engine-patched probe connection

```

1 assign rvfic_valid = core.rvfic_probe_inst.probe_rvfic_valid;

```

At this point the signal path is complete. At simulation or formal verification time, `rvfic_valid` carries the value of the original expression evaluated inside the DUV, exposed through a non-intrusive bind probe and presented through the wrapper’s uniform interface.

4.3.9 Limitations and Comparison

The map specification has limitations. Cores with out-of-order completion require additional wrapper logic. The map can rename and tap signals but cannot restructure data, so a core that reports memory access information in a format incompatible with RVFI requires manual wrapper code. Expressions are inserted as is into generated SystemVerilog, meaning complex expressions may cause timing issues that RVFIC cannot detect. Finally, RVFIC cannot automatically locate signals within an unknown core, so creating a map requires understanding the DUV’s internal structure.

The riscv-formal project [3] takes a different approach. Cores must expose a fixed set of RVFI signals with predetermined names and semantics. Cores that deviate from this interface require RTL modifications. RVFIC inverts this relationship by adapting to each core’s existing structure. This matters because modifying third-party IP is often impractical or sometimes prohibited, the same core may need different mappings for different verification scenarios, and cores under active development may change their internal structure frequently. The trade-off is that RVFIC requires a map file for each core, whereas riscv-formal can immediately verify any core exposing the standard interface. However, this cost is paid once per core and then reused for all later verification work.

4.4 RISC-V Golden Model (RVGM)

The second component of the verification framework is a parameterizable reference model that captures the operational semantics of the RISC-V ISA. While RVFIC (Section 4.3) exposes the DUV’s architectural state through a uniform interface, it does not define what constitutes correct behavior. A verification flow needs a golden model that specifies, for each instruction, how the architectural state evolves. Operational semantics provides exactly this through a set of rules describing the effect of each instruction on the machine state. Given the current program counter, register file contents, and memory, the semantics defines the values these components must hold

after the instruction executes. By encoding these rules as synthesizable hardware, the golden model becomes an executable specification that can be compared against a DUV cycle by cycle.

RVGM is a synthesizable RISC-V golden model implemented in Scala/Chisel. It models a minimal in-order CPU that retires at most one instruction per cycle, maintains a program counter and a 32-entry integer register file, and expects memory as an external interface. The model is deliberately simple. It does not implement a full ISA emulator but only models the architectural effects visible through RVFI signals. Every computation is combinational within a single cycle, and state updates occur only when an instruction retires.

RVGM supports parameterized ISA configurations. The same generator can produce a model for RV32I, RV64I, or any combination of the M (multiply/divide), W (RV64 32-bit operations), C (compressed instructions), and CSR extensions. Changing a configuration flag adds or removes the corresponding instruction semantics at Chisel elaboration time, so the emitted SystemVerilog contains only the logic required for the selected ISA subset. This matters for formal verification because unused logic increases the state space that the formal tool must explore.

RVGM exposes its outputs through an RVFI-compatible interface. The signal names, widths, and semantics match those produced by RVFIC wrappers, so the verification framework can compare DUV and golden model outputs signal by signal. Optional CSR trace ports extend the interface when CSR verification is enabled, using the same conditional elaboration mechanism.

Figure 4.7 shows the overall dataflow. An instruction word enters the model, passes through optional decompression and register read stages, and is then dispatched to the SpecAggregator, which broadcasts it to all enabled per-instruction specification modules. The single matching module computes the expected architectural effects, and the golden model updates its state registers (program counter, register file, and retirement order counter) only when the instruction retires. The results are exposed through RVFI-compatible output bundles. The following subsections describe each component in detail.

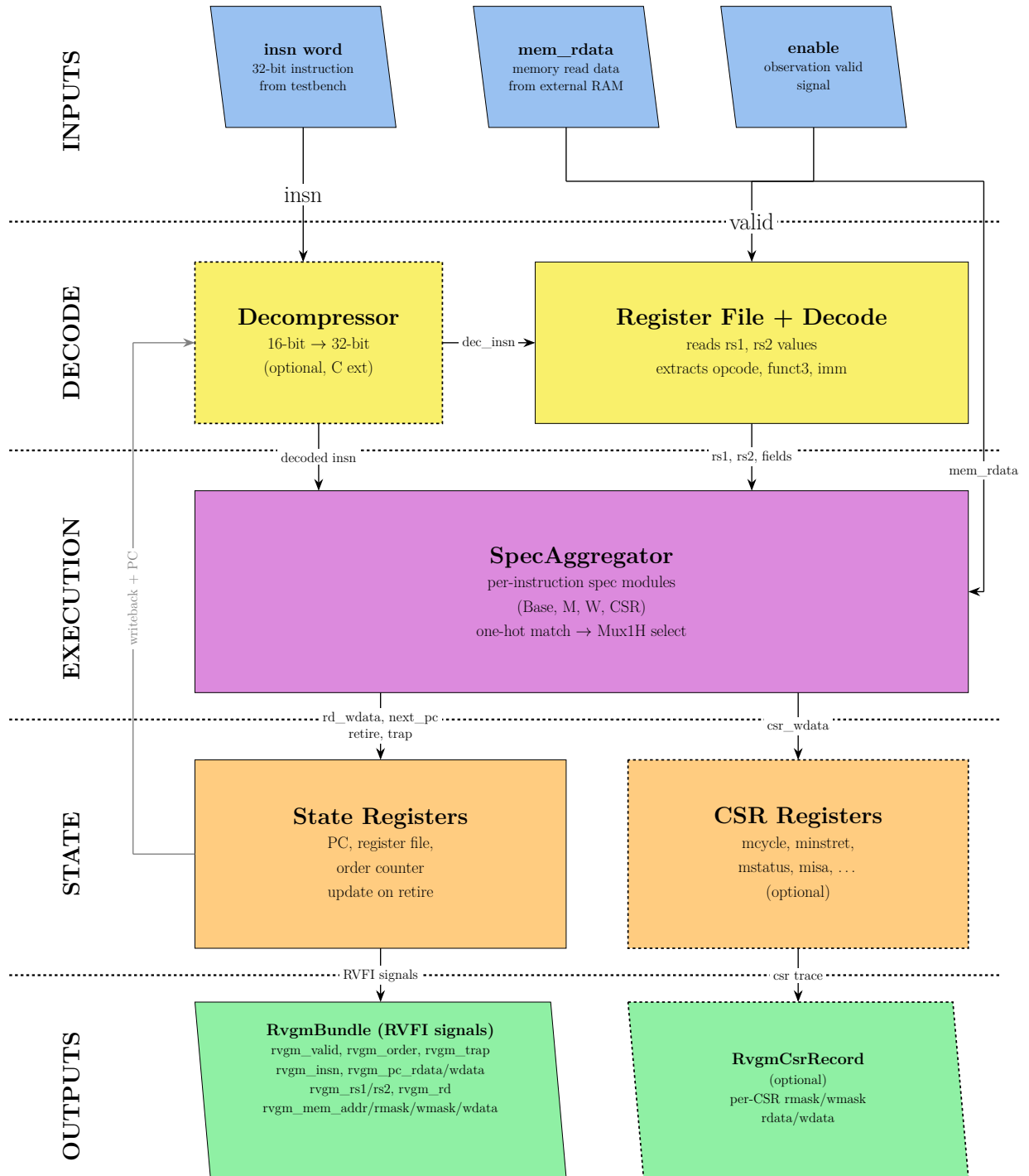


Figure 4.7: RVGM golden model architecture (Dashed blocks are conditionally instantiated based on the ISA configuration).

4.4.1 Configuration

The RVGM configuration (`rvgmconfig.scala`) contains `RvgmConfig` case class which collects all elaboration time parameters that shape the golden model. It is the object that all components consume, ensuring consistency across the generated design. The configuration is divided into following groups:

- **ISA parameters:** Controls the register width and which RISC-V extensions to enable.
 - `XLEN` (RV32 or RV64)
 - boolean flags (`enableM`, `enableW`, `enableC`, and `enableCSR`)
- **CSR trace flags:** Controls whether the golden model instantiates the corresponding CSR register and exposes its trace ports.
 - nine independent booleans (`traceCsrMcycle`, `traceCsrMinstret`, `traceCsrMvendorid`, `traceCsrMarchid`, `traceCsrMimpid`, `traceCsrMhartid`, `traceCsrMstatus`, `traceCsrMisa`, `traceCsrMtvec`)
- **CSR value parameters:** Allows the golden model to match the DUV's CSR initialization, which differs across cores.
 - initial values and timing offsets for the counter CSRs (`csrMcycleStartValue`, `csrMcycleStartDelay`, and their `minstret` counterparts)
 - reset values for `mstatus`, and constant values for the identification CSRs (`csrVendorId`, `csrArchId`, `csrImpId`, `csrHartId`, `csrMtvecBase`)

Configuration values are supplied through environment variables. Read variables such as `RVGM_XLEN`, `RVGM_ENABLE_M`, and `RVGM_ENABLE_CSR` construct the corresponding `RvgmConfig`. CSR tracing supports a profile mechanism. Setting `RVGM_TRACE_PROFILE` to `minimal` enables only `mcycle` and `minstret`, while `machine_full` or `all` enables all nine trace flags. Table 4.2 summarizes the key configuration parameters, their types, and default values.

Table 4.2: Key parameters in `RvgmConfig`.

Parameter	Type	Default	Description
<code>xlen</code>	Int	64	Register width (32 or 64)
<code>enableM</code>	Boolean	false	M extension (mul/div)
<code>enableW</code>	Boolean	false	W extension (RV64 32-bit ops)
<code>enableC</code>	Boolean	false	C extension (compressed)
<code>enableCSR</code>	Boolean	false	CSR instruction semantics
<code>orderBits</code>	Int	64	Retirement counter width
<code>pcStart</code>	Long	0	Initial program counter
<code>memBytes</code>	Int	4096	Testbench memory size

4.4.2 Interface

The golden model communicates its results through two Chisel bundles that define the shape of the emitted SystemVerilog ports.

`RvgmBundle` (`rvgmbundle.scala`) is the primary output interface. It contains all RVFI base signals and memory signals as fixed output ports. The bundle is parameterized by `RvgmConfigIfc`, which carries the XLEN and order counter width. Table 4.3 lists the signals and their widths.

Table 4.3: Signals in `RvgmBundle`.

Signal	Width	Description
<code>rvgm_valid</code>	1	Instruction retired this cycle
<code>rvgm_trap</code>	1	Trap indicator (illegal insn or CSR trap)
<code>rvgm_order</code>	<code>orderBits</code>	Monotonic retirement counter
<code>rvgm_insn</code>	32	Instruction word (original for compressed)
<code>rvgm_pc_rdata</code>	XLEN	PC at retirement
<code>rvgm_pc_wdata</code>	XLEN	Next PC
<code>rvgm_rs1_addr</code>	5	Source register 1 address
<code>rvgm_rs2_addr</code>	5	Source register 2 address
<code>rvgm_rs1_rdata</code>	XLEN	Source register 1 data
<code>rvgm_rs2_rdata</code>	XLEN	Source register 2 data
<code>rvgm_rd_addr</code>	5	Destination register address
<code>rvgm_rd_wdata</code>	XLEN	Destination register writeback value
<code>rvgm_mem_addr</code>	XLEN	Memory effective address
<code>rvgm_mem_rmask</code>	XLEN/8	Memory read byte mask
<code>rvgm_mem_wmask</code>	XLEN/8	Memory write byte mask
<code>rvgm_mem_rdata</code>	XLEN	Memory read data
<code>rvgm_mem_wdata</code>	XLEN	Memory write data

Unlike RVFIC’s `RvficBundle`, where memory signals are optional, `RvgmBundle` always includes memory signals. This is because the golden model always computes memory access intent. Including them unconditionally simplifies the formal comparison because every signal that RVFIC can expose has a corresponding golden model output.

`RvgmCsrRecord` (`rvgmcsrrecord.scala`) is the second output bundle, handling optional CSR trace ports. It is implemented as a Chisel `Record`. It’s a dynamically-shaped bundle whose fields are determined at elaboration time. The record is parameterized by `RvgmCsrConfig`, which contains a boolean flag for each supported CSR. When a flag is false, the corresponding ports are not added to the record at all, and no SystemVerilog ports are emitted.

For each enabled CSR, the record creates four ports:

- `<csr>_rmask`: read mask, indicating which bits were read.
- `<csr>_wmask`: write mask, indicating which bits were written.
- `<csr>_rdata`: the CSR value before the instruction.
- `<csr>_wdata`: the value written to the CSR.

The counter CSRs (`mcycle`, `minstret`) use 64-bit ports regardless of XLEN, since the counters are architecturally 64 bits wide. All other CSRs use XLEN-wide ports.

This conditional port generation mirrors RVFIC’s approach of adapting the interface shape to the configuration. When RVGM and RVFIC are used together in a formal environment, both sides generate the same set of CSR ports based on the same trace flags, ensuring that the comparison covers exactly the CSRs the user requested.

4.4.3 Specification Modules

RVGM models each RISC-V instruction as an independent Chisel module. This one-module-per-instruction pattern isolates instruction semantics so that, for example, a bug in the `ADDI` implementation cannot affect `SLL` because they share no mutable state. The pattern also makes the model extensible, since adding a new instruction requires writing one new module and registering it in the appropriate operations list without modifying existing code.

Every per-instruction module extends the base class `SpecModule`, which provides a standardized IO consisting of `SpecInputs` for the observed state at retirement time and `SpecOutputs` for the computed effects of the instruction. The base class also supplies default output values where `spec_valid` defaults to `false`, register addresses and memory masks default to zero, and the next program counter defaults to the current PC plus the instruction length. A module only needs to override the outputs that its instruction actually drives.

Tables 4.4 and 4.5 summarize the signals in both bundles.

Table 4.4: `SpecInputs` — observed state at retirement time.

<code>obs_valid</code>	Golden model is actively retiring.
<code>obs_insn</code>	32-bit instruction word (after decompression).
<code>obs_insn_len</code>	Instruction length in bytes (2 or 4); used for default PC advancement.
<code>obs_pc</code>	Current program counter.
<code>obs_rs1</code> , <code>obs_rs2</code>	Source register values from the register file.
<code>obs_mem_rdata</code>	Memory read data (loads).
<code>obs_order_lo</code> , <code>obs_minstret</code>	<code>obs_mcycle</code> , Counter values for CSR modules.

Table 4.5: SpecOutputs — computed effects of the instruction.

<code>spec_valid</code>	Asserted when this module matches the current instruction; exactly one module asserts per cycle.
<code>spec_rs1_addr</code> , <code>spec_rs2_addr</code>	Source register addresses, forwarded to RVFI for read-consistency checking.
<code>spec_rd_addr</code> , <code>spec_rd_wdata</code>	Destination register and write-back value.
<code>spec_pc_wdata</code>	Next program counter.
<code>spec_mem_addr</code> , <code>spec_mem_rmask</code> , <code>spec_mem_wmask</code> , <code>spec_mem_wdata</code>	Memory access intent (loads and stores).

Both bundles are parameterized by the XLEN value. This ensures that all data widths (registers, addresses, memory masks) adapt automatically to the configured register width.

A typical spec module follows a three-step pattern: extract instruction fields, check the opcode and function bits to determine whether this module matches, and compute the output values. The following example shows the ADDI module:

Example 4.4.1: ADDI specification module (`rvfi_insn_addi`)

```

1 class rvfi_insn_addi(xlen: Int) extends SpecModule(SpecCfg(xlen)) {
2   private val insn = io.in.obs_insn
3   private val rs1  = insn(19,15)      // source register index
4   private val funct3 = insn(14,12)    // ALU function selector
5   private val rd    = insn(11,7)     // destination register index
6   private val opcode = insn(6,0)
7   // I-type immediate: sign-extend insn[31:20] to XLEN bits
8   private val imm12 = insn(31,20)
9   private val imm = {
10     val sign = imm12(11)
11     Cat(Fill(cfg.xlen - 12, sign), imm12)
12   }
13   // Match condition: ADDI is funct3=0x0, opcode=0x13
14   val isAddi = io.in.obs_valid &&
15               (funct3 === "b000".U) &&
16               (opcode === "b0010011".U)
17   val result = io.in.obs_rs1 + imm    // rs1 + sign_ext(imm)
18   io.out.spec_valid := isAddi
19   io.out.spec_rs1_addr := rs1
20   io.out.spec_rs2_addr := 0.U
21   io.out.spec_rd_addr := rd
22   io.out.spec_rd_wdata := Mux(rd.orR, result, 0.U)
23   io.out.spec_pc_wdata := io.in.obs_pc + io.in.obs_insn_len
24 }

```

The module extracts the register addresses and the 12-bit immediate from the instruction word, checks that the opcode and `funct3` field match ADDI, sign-extends the immediate to XLEN bits,

and produces the sum of `rs1` and the immediate as the writeback value. The next program counter advances by the instruction length (the default from `SpecModule`). Memory outputs remain at their defaults (zero masks), since `ADDI` does not access memory.

4.4.4 Golden Model

The `GoldenModel` (`goldenmodel.scala`) class is the top-level Chisel module that implements the reference RISC-V core. It takes `RvgmConfig` instance (Section 4.4.1) at elaboration time and produces a synthesizable module whose behavior is fully determined by the ISA configuration. The module maintains three pieces of architectural state:

- A **program counter** (`pcReg`): XLEN bits wide, initialized to zero (or a configurable start address).
- A **register file** (`regs`): Consists of 32 registers of XLEN bits each. Register `x0` is hardwired to zero by masking reads and writes through the `orR` guard: any access to address zero returns zero, and writes to address zero are suppressed.
- A **retirement order counter** (`order`): Has a configurable width defaulting to 64. It increments by one on every retiring instruction and holds its value otherwise.

The module's IO consists of three inputs and two output bundles. The inputs are an `enable` signal that gates instruction retirement, a 32-bit `insn` word, and an XLEN-wide `mem_rdata` line providing memory read data from the external environment. The outputs are an `RvgmBundle` (Section 4.4.2) carrying the RVFI signals, and optional CSR trace ports. Execution proceeds in four combinational phases within a single cycle:

1. **Decompression:** If the C extension is enabled, the incoming instruction passes through decompression, which converts 16-bit compressed encodings into their canonical 32-bit equivalents and sets the instruction length to 2 or 4 bytes. If decompression fails, the instruction is marked illegal.
2. **Register fetch:** The source register addresses `rs1` and `rs2` are extracted from the canonical instruction word. The corresponding register values are read from the register file, with the `x0` guard applied.
3. **Semantic dispatch:** The instruction word and observed state are forwarded to the `SpecAggregator` (Section 4.4.5), which selects the matching per-instruction module and returns the computed effects: destination register value, next program counter, and memory access intent.
4. **State update:** A `retire` signal is asserted when the aggregator matches the instruction, when the instruction is an illegal trap, or when it is a CSR instruction with CSR support disabled. Only when `retire` is true does the model update its state: the program counter advances to the computed next value, the order counter increments, and the destination register receives the computed writeback value.

The separation between combinational computation and sequential state update is central to the model's correctness argument. The spec modules (Section 4.4.3) that observe state and produce

outputs are purely combinational. They never modify registers or memory directly. The golden model applies their results to its state only under the **retire** guard. This makes the model's behavior straightforward to verify. Each cycle either retires one instruction and updates state accordingly, or does nothing.

Memory is handled through an externalized interface. The golden model computes the effective address and byte masks for loads and stores, but the actual memory array lives outside the model. This design keeps the golden model free of memory-related side effects and makes it compatible with both simulation testbenches and formal environments where memory is modeled differently.

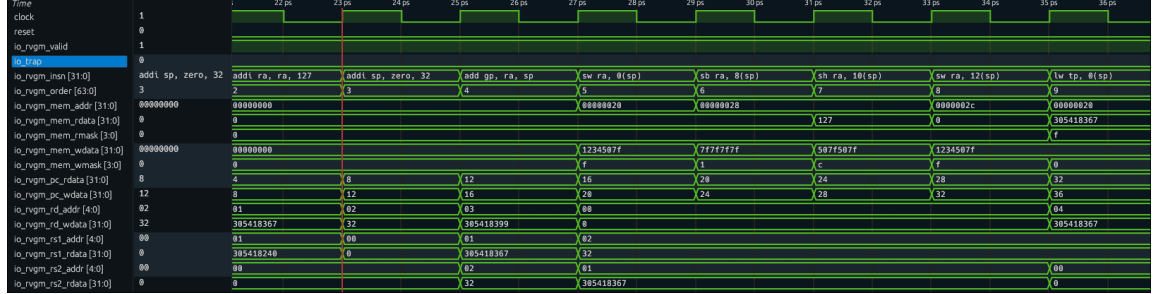


Figure 4.8: RVGM simulation waveform showing instruction retirement.

When CSR tracing is enabled, the golden model implements CSR semantics directly within `GoldenModel`. `mcycle`, `minstret` and `mstatus`. Each CSR is conditionally instantiated based on its trace flag in the configuration. The `mcycle` counter increments every cycle regardless of retirement, while `minstret` increments only when an instruction retires. Both counters respect the `mcountinhibit` register, which allows selective suppression of counter increments. CSR read values are fed back into the register file writeback path when a CSR instruction targets a destination register, so that sequences like `CSRRS x1, mcycle, x0` can correctly write the current counter value into `x1`.

4.4.5 Specification Aggregator

The `SpecAggregator` (`specaggregator.scala`) is the central decoder and dispatcher of the golden model. It connects the observed state from the golden model to all enabled spec modules and selects the single module that matches the current instruction.

The aggregator is parameterized by the XLEN value and three extension flags: `enableM`, `enableW`, and `enableCSR`. At elaboration time, it constructs the set of active spec modules by concatenating four groups:

- **Base modules:** always instantiated. These cover the RV32I instruction set. When XLEN is 64, additional RV64I-only instructions (LWU, LD, SD) are appended via `BaseOps.xlen64`.
- **M extension modules:** instantiated only when `enableM` is true. These add the eight multiply/divide/remainder instructions.
- **W extension modules:** instantiated only when `enableW` is true and XLEN is 64. These add the nine RV64W 32-bit arithmetic instructions.

- **CSR modules:** instantiated only when `enableCSR` is true. These add the six CSR read/write instructions.

Once the module list is assembled, the aggregator broadcasts all observed inputs to every module in parallel. Each module independently evaluates its match condition (`spec_valid`) and computes its outputs. The aggregator then combines the results using a one-hot multiplexer (`Mux1H`): each output field is selected from the module whose `spec_valid` is asserted. When no module matches, all outputs default to zero (or the current PC for `spec_pc_wdata`).

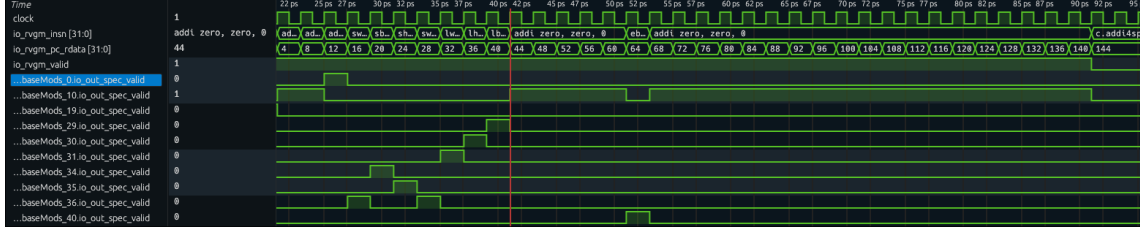


Figure 4.9: Waveform showing the `SpecAggregator` one-hot selection. Each cycle, exactly one spec module asserts `spec_valid`, and the aggregator forwards that module's outputs.

The aggregator does not hold any state. It is a purely combinational circuit that transforms observed inputs into per-instruction effects. All sequential state such as the program counter, register file, and order counter is managed by the `GoldenModel` (Section 4.4.4), which reads the aggregator's outputs and applies them under the retirement guard. This separation ensures that the aggregator can be reasoned about independently. given the same inputs, it always produces the same outputs.

4.4.6 Limitations and Comparison

RVGM models a subset of the RISC-V architecture sufficient for verifying the integer pipeline of in-order cores. Several aspects of the full architecture such as Interrupts and exceptions, virtual memory, privilege modes, floating point and vector extensions and out-of-order execution are not modelled.

These limitations are deliberate. The model is designed for formal verification of integer pipelines, where the formal tool explores bounded instruction sequences. In this context, a simpler model which has fewer state bits for the solver to track is preferable. It produces smaller SystemVerilog, and is easier to verify for correctness.

RVGM's CSR semantics deserve particular mention. An implementation of CSR support is present in the golden model, but it remains incomplete and exhibits several correctness issues. Although basic CSR handling exists, the behavior is not yet reliable and does not consistently match the DUV across tested cases. This is largely due to the implementation-specific nature of CSR behavior. Achieving full agreement would require reproducing substantial parts of the CSR implementation, which conflicts with the RVGM's intentionally simple design philosophy.

Other RISC-V reference models take different approaches. Spike⁴ is a widely used instruction set simulator that models the full architecture including privilege modes, virtual memory, and all standard extensions. It serves as a canonical functional reference but produces software traces

⁴<https://github.com/riscv-software-src/riscv-isa-sim>

rather than synthesizable hardware. The Sail model is a formal specification written in the Sail language, which can generate both executable simulators and theorem prover definitions. Both are comprehensive but not directly usable as synthesizable golden models in a hardware formal verification flow.

The riscv-formal project [3] takes yet another approach. Instead of a monolithic golden model, it defines per-instruction formal checks that verify each instruction independently against the RVFI interface. This avoids the need for a golden model entirely but requires each check to be written and maintained separately, and it cannot verify inter-instruction properties (such as register file consistency across sequences of instructions) without additional infrastructure.

RVGM occupies a middle ground. It is synthesizable, so not only it can be instantiated alongside a DUV in a formal environment and compared signal by signal, but also it can be used in simulation. It is parameterizable, so one generator covers multiple ISA configurations. It is modular, so individual instruction semantics can be reviewed, tested, and extended independently. The trade-off is reduced architectural coverage compared to Spike or Sail, which is acceptable for the integer pipeline verification use case this framework targets.

Table 4.6 shows how the generated artifact sizes change as extensions are incrementally enabled. The generator LOC remains constant at 3405 lines across all configurations, as the full Scala source is always present. The generated LOC grows with each extension: enabling M adds 446 lines for multiply/divide specifications, CSR adds 325 lines, W adds 459 lines for 32-bit operations on RV64, and C adds 279 lines for decompression logic. The number of generated artifacts increases from 47 to 70 files as more per-instruction specification modules are emitted.

Table 4.6: RVGM generator versus generated artifact LOC.

Step	Configuration	Generator LOC	Artifacts	Generated LOC	Delta
1	RV64I (base)	3405	47	2584	–
2	+ M	3405	55	3030	+446
3	+ CSR	3405	61	3355	+325
4	+ W	3405	70	3814	+459
5	+ C	3405	70	4093	+279

4.5 RISC-V Automatic Verification Framework (RVAVF)

RVAVF is the top-level integration that combines RVFIC (Section 4.3) and RVGM (Section 4.4) into a unified formal verification flow. While RVFIC exposes the DUV’s architectural state through a uniform interface, and RVGM provides a reference specification of correct behavior, neither component alone constitutes a verification environment. RVAVF bridges this gap by generating SystemVerilog assertions that compare the two RVFI interfaces signal-by-signal at instruction retirement boundaries.

The framework operates by instantiating both the RVFIC-wrapped DUV and an RVGM-based golden model in a shared top-level harness. Both modules receive the same instruction stream and produce RVFI outputs describing the architectural effects of each retired instruction. RVAVF generates assertions that check whether the DUV’s outputs match the golden model’s outputs

for every retiring instruction. When the formal tool finds a counterexample, it has identified an instruction sequence where the DUV deviates from the specification.

The verification flow is fully automated. Given a core preset or a custom configuration, RVAVF invokes the RVFIC and RVGM generators, produces assertion modules and a top-level harness, stages all files into a workspace, and generates a TCL script for JasperGold. The user runs a single make target and receives either a proof that all assertions hold within the bounded depth, or a counterexample trace showing the first divergence.

4.5.1 Configuration

RVAVF unifies the configuration mechanisms of RVFIC and RVGM into a single environment-driven system. The `RvavfConfig` case class collects parameters that control assertion generation, top-level harness construction, and formal script generation.

Example 4.5.1: RvavfConfig case class collecting top-level parameters

```

1 case class RvavfConfig(
2   name: String,           // Run label (e.g., "ibex", "picorv32")
3   xlen: Int,              // Register width (32 or 64)
4   enableCsr: Boolean,     // Enable CSR signal comparison
5   enableMemorySignals: Boolean, // Enable memory signal comparison
6   traceProfile: String,   // CSR trace preset (none, minimal, machine_full)
7   cexTraces: Boolean,     // Enable JasperGold counterexample traces
8   goldenModelName: String, // Golden model identifier ("rvgm")
9   duvName: String,        // DUV identifier (e.g., "ibex", "picorv32")
10  duvModule: String       // DUV wrapper module name
11 )

```

Configuration values are read from environment variables at generator invocation time. The `RVAVF_*` variables control the top-level flow, while `RVGM_*` and `RVFIC_*` variables are forwarded to the respective submodule generators. This layered approach allows users to configure all three components through a single interface while preserving the ability to override submodule-specific settings. Table 4.7 summarizes the key configuration parameters.

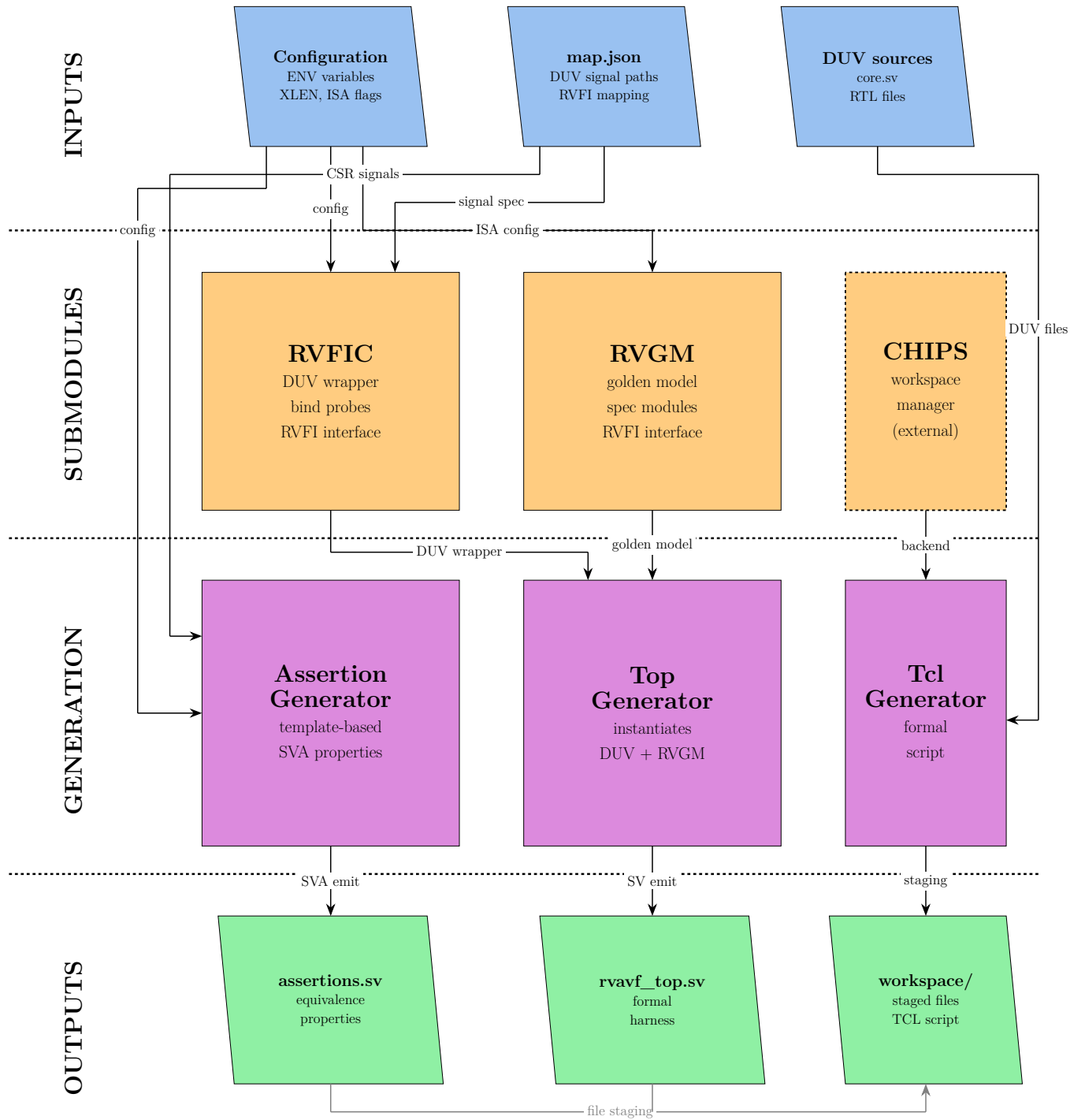


Figure 4.10: RVAVF formal verification flow architecture.

Table 4.7: Key RVAVF configuration parameters and their defaults.

Parameter	Type	Default
RVAVF_NAME	String	picorv32
RVAVF_XLEN	Integer	32
RVAVF_ENABLE_CSR	Boolean	true
RVAVF_ENABLE_MEM	Boolean	true
RVAVF_TRACE_PROFILE	String	none
RVAVF_DUV_NAME	String	picorv32
RVAVF_DUV_MODULE	String	RvficWrapper
RVAVF_MAX_LATENCY	Integer	32
RVAVF_CORE	String	(none)

For common verification targets, RVAVF provides predefined core presets that bundle the appropriate map file, DUV source paths, and feature flags. Setting `RVAVF_CORE` to a preset name (such as `ibex` or `picorv32`) automatically configures all dependent variables. This reduces the configuration burden for standard cores and ensures consistency between the RVFIC wrapper and the RVGM golden model.

RVAVF uses a two-level system for controlling assertions. Global flags such as `RVAVF_ENABLE_CSR` and `RVAVF_ENABLE_MEM` enable or disable entire categories of assertions. Individual assertion enables such as `RVAVF_ENABLE_ASSERT_MATCH_INSN_IBEX` provide fine-grained control for debugging specific failures. When a global flag is disabled, the corresponding assertions are excluded from generation entirely. When an individual enable is set to false, the assertion is generated but wrapped in a conditional generate block that the formal tool skips.

4.5.2 Assertion Generation

Unlike RVFIC and RVGM, which use Chisel hardware generators to produce SystemVerilog through the FIRRTL compilation flow, the `AssertionGenerator` is a template-based code generator. It constructs SystemVerilog assertion modules by interpolating configuration values and signal names into predefined string templates. This approach is well-suited for generating SVA properties because assertions are declarative constructs that do not benefit from Chisel’s hardware abstraction features.

The generator defines assertion specifications as Scala case classes containing a name, an SVA body template, and enable parameters. The body templates substitute signal names and configuration values at generation time.

Example 4.5.2: Template-based assertion specification in Scala

```

1 case class AssertionSpec(
2   name: String,
3   body: String,
4   enableDefault: String = "1'b1"
5 ) {
6   val enableParam: String = s"RVAVF_ENABLE_${name.toUpperCase}"
7 }
8 val matchAntecedent = "rvfic_spec_valid"
9 val duvMatchEnable = "rvfic_valid && !rvfic_trap && !rvfic_spec_trap"
10 AssertionSpec(
11   name = "assert_match_insn",
12   body = s"""
13     | assert_match_insn: assert property (@(posedge clk) disable iff (rst)
14     |     ($matchAntecedent && $duvMatchEnable)
15     |     |-> (rvfic_insn == rvfic_spec_insn)
16     | );
17     |""".stripMargin
18 )

```

Each `AssertionSpec` is rendered into a conditional generate block, allowing individual assertions to be enabled or disabled via module parameters without regenerating the source. The generator collects all specifications, applies configuration-specific filtering, and emits a complete SystemVerilog module.

Example 4.5.3: Generated assertion module

```

1 module rvavf_base_assertions #(
2   parameter int XLEN = 32,
3   parameter bit RVAVF_ENABLE_ASSERT_MATCH_INSN = 1'b1,
4   parameter bit RVAVF_ENABLE_ASSERT_SPEC_ORDER_INCREMENTS = 1'b1,
5   // ... additional enable parameters
6 ) (
7   input logic clk, rst,
8   input logic rvfic_spec_valid, rvfic_valid,
9   input logic [31:0] rvfic_spec_insn, rvfic_insn,
10  // ... additional signal ports
11 );
12 generate
13   if (RVAVF_ENABLE_ASSERT_MATCH_INSN) begin : gen_assert_match_insn
14     assert_match_insn: assert property (@(posedge clk) disable iff (rst)
15       (rvfic_spec_valid && rvfic_valid && !rvfic_trap && !rvfic_spec_trap)
16       |-> (rvfic_insn == rvfic_spec_insn)
17     );
18   end
19 endgenerate
20 endmodule

```

The generated module includes over 40 assertions and assumptions organized into categories such as equivalence checking (matching DUV outputs against the golden model), self-consistency (verifying internal invariants), and environment constraints (restricting the input space for tractable proofs).

Each assertion follows the SVA implication pattern, which corresponds to the LTL formula $\mathbf{G}(\textit{antecedent} \rightarrow \textit{consequent})$. The $\mathbf{G}$ (globally) operator is implicit in SVA concurrent assertions: the property is evaluated at every clock cycle. Table 4.8 lists all generated assertions with their corresponding LTL formulas, using the following notation:

- *spec_v*, *duv_v*: spec (golden model) or DUV retired an instruction (valid signal asserted)
- *ret*: successful retirement ($\textit{spec_v} \wedge \textit{duv_v} \wedge \neg \textit{trap}$)
- *trap*: trap signal asserted (exception or interrupt)
- *1st*, *2nd*: first or second retired instruction (order counter equals 0 or 1)
- *nc*, *c*: non-compressed or compressed instruction encoding ($\textit{insn}[1:0] == 2'b11$ or $!= 2'b11$)
- *wr_rd*: instruction writes to destination register ($\textit{rd_addr} \neq 0$)
- *use_rs1*, *use_rs2*: instruction reads source register 1 or 2
- *is_mem*, *is_ld*, *is_st*: memory operation, load, or store instruction
- Subscripts *_d* and *_s*: DUV and spec (golden model) signals respectively
- *ord*, *ord_p*: order counter value and previous cycle's order counter value
- *pc_r*, *pc_w*: program counter read data (before execution) and write data (after execution)
- *rd_a*, *rs1_a*, *rs2_a*: destination and source register addresses
- *rd*, *rs1*, *rs2*: destination and source register data values
- *ma*, *msk*, *mrd*, *mwd*: memory address, masks, read data, and write data

- *align*: required PC alignment (4 for non-compressed, 2 for compressed)

Table 4.8: Generated SVA assertions with LTL semantics

Assertion	LTL Formula	Description
<i>Retirement & Basic Signals</i>		
<code>match_valid</code>	$\mathbf{G}(spec_v \rightarrow duv_v)$	DUV retires when spec retires
<code>match_order</code>	$\mathbf{G}(ret \rightarrow ord_d = ord_s)$	Order counters match
<code>match_insn</code>	$\mathbf{G}(ret \rightarrow insn_d = insn_s)$	Instruction words match
<i>Program Counter</i>		
<code>match_pc_rdata_boot</code>	$\mathbf{G}((ret \wedge 1st) \rightarrow pc_r_d = pc_r_s)$	PC read matches (1st insn)
<code>match_pc_rdata_second</code>	$\mathbf{G}((ret \wedge 2nd) \rightarrow pc_r_d = pc_r_s)$	PC read matches (2nd insn)
<code>match_pc_wdata_boot</code>	$\mathbf{G}((ret \wedge 1st) \rightarrow pc_w_d = pc_w_s)$	PC write matches (1st insn)
<code>match_pc_wdata_second</code>	$\mathbf{G}((ret \wedge 2nd) \rightarrow pc_w_d = pc_w_s)$	PC write matches (2nd insn)
<i>Destination Register (rd)</i>		
<code>match_gpr_rd_addr</code>	$\mathbf{G}((ret \wedge wr_rd) \rightarrow rd_a_d = rd_a_s)$	rd address matches
<code>match_gpr_rd_wdata_boot</code>	$\mathbf{G}((ret \wedge 1st \wedge wr_rd) \rightarrow rd_d = rd_s)$	rd data matches (1st insn)
<code>match_gpr_rd_wdata_second</code>	$\mathbf{G}((ret \wedge 2nd \wedge wr_rd) \rightarrow rd_d = rd_s)$	rd data matches (2nd insn)
<i>Source Register 1 (rs1)</i>		
<code>match_gpr_rs1_addr_nc</code>	$\mathbf{G}((ret \wedge nc \wedge use_rs1) \rightarrow rs1_a_d = rs1_a_s)$	rs1 addr matches (nc)
<code>match_gpr_rs1_addr_c</code>	$\mathbf{G}((ret \wedge c \wedge use_rs1) \rightarrow rs1_a_d = rs1_a_s)$	rs1 addr matches (c)
<code>match_gpr_rs1_rdata_nc_boot</code>	$\mathbf{G}((ret \wedge 1st \wedge nc \wedge use_rs1) \rightarrow rs1_d = rs1_s)$	rs1 data matches (nc, 1st)
<code>match_gpr_rs1_rdata_nc_second</code>	$\mathbf{G}((ret \wedge 2nd \wedge nc \wedge use_rs1) \rightarrow rs1_d = rs1_s)$	rs1 data matches (nc, 2nd)
<code>match_gpr_rs1_rdata_c_boot</code>	$\mathbf{G}((ret \wedge 1st \wedge c \wedge use_rs1) \rightarrow rs1_d = rs1_s)$	rs1 data matches (c, 1st)
<code>match_gpr_rs1_rdata_c_second</code>	$\mathbf{G}((ret \wedge 2nd \wedge c \wedge use_rs1) \rightarrow rs1_d = rs1_s)$	rs1 data matches (c, 2nd)
<i>Source Register 2 (rs2)</i>		

Continued on next page

Table 4.8 – continued from previous page

Assertion	LTL Formula	Description
match_gpr_rs2_addr_nc	$\mathbf{G}((ret \wedge nc \wedge use_rs2) \rightarrow rs2_a_d = rs2_a_s)$	rs2 addr matches (nc)
match_gpr_rs2_addr_c	$\mathbf{G}((ret \wedge c \wedge use_rs2) \rightarrow rs2_a_d = rs2_a_s)$	rs2 addr matches (c)
match_gpr_rs2_rdata_nc_boot	$\mathbf{G}((ret \wedge 1st \wedge nc \wedge use_rs2) \rightarrow rs2_d = rs2_s)$	rs2 data matches (nc, 1st)
match_gpr_rs2_rdata_nc_second	$\mathbf{G}((ret \wedge 2nd \wedge nc \wedge use_rs2) \rightarrow rs2_d = rs2_s)$	rs2 data matches (nc, 2nd)
match_gpr_rs2_rdata_c_boot	$\mathbf{G}((ret \wedge 1st \wedge c \wedge use_rs2) \rightarrow rs2_d = rs2_s)$	rs2 data matches (c, 1st)
match_gpr_rs2_rdata_c_second	$\mathbf{G}((ret \wedge 2nd \wedge c \wedge use_rs2) \rightarrow rs2_d = rs2_s)$	rs2 data matches (c, 2nd)
<i>Memory (conditional)</i>		
match_mem_addr	$\mathbf{G}((ret \wedge is_mem) \rightarrow ma_d = ma_s)$	Memory address matches
match_mem_masks	$\mathbf{G}((ret \wedge is_mem) \rightarrow msk_d = msk_s)$	Memory masks match
match_mem_rdata	$\mathbf{G}((ret \wedge is_ld) \rightarrow mrd_d = mrd_s)$	Memory read data matches
match_mem_wdata	$\mathbf{G}((ret \wedge is_st) \rightarrow mwd_d = mwd_s)$	Memory write data matches
<i>Order Counter Monotonicity</i>		
spec_order_increments	$\mathbf{G}((spec_v \wedge \neg trap) \rightarrow \mathbf{X}(ord = ord_p + 1))$	Spec order incr on retire
spec_order_stable	$\mathbf{G}(\neg spec_v \rightarrow \mathbf{X}(ord = ord_p))$	Spec order stable when idle
duv_order_increments	$\mathbf{G}((duv_v \wedge \neg trap) \rightarrow \mathbf{X}(ord = ord_p + 1))$	DUV order incr on retire
duv_order_stable	$\mathbf{G}(\neg duv_v \rightarrow \mathbf{X}(ord = ord_p))$	DUV order stable when idle
<i>PC Alignment</i>		
spec_pc_aligned	$\mathbf{G}(spec_v \rightarrow (pc_s \bmod align = 0))$	Spec PC is aligned
duv_pc_aligned	$\mathbf{G}(duv_v \rightarrow (pc_d \bmod align = 0))$	DUV PC is aligned

The fundamental equivalence assertion checks that when the golden model retires an instruction, the DUV also retires:

$$\mathbf{G}(spec_valid \rightarrow duv_valid)$$

Example 4.5.4: Retirement matching assertion

```

1 assert_match_valid: assert property (@(posedge clk) disable iff (rst)
2   (rvfic_spec_valid) |-> rvfic_valid
3 );

```

When both models retire without trapping, their architectural outputs must match. Let *retire* denote the conjunction $spec_valid \wedge duv_valid \wedge \neg trap$:

$$\mathbf{G}(\text{retire} \rightarrow (duv_insn = spec_insn))$$

Example 4.5.5: Instruction word matching assertion

```

1 assert_match_insn: assert property (@(posedge clk) disable iff (rst)
2   (rvfic_spec_valid && rvfic_valid && !rvfic_trap && !rvfic_spec_trap)
3   |-> (rvfic_insn == rvfic_spec_insn)
4 );

```

Similar assertions compare program counter values, register file addresses and data, and memory access signals when applicable.

The RVFI order counter must increment by exactly one on each successful retirement and remain stable otherwise. Using the **X** (next) operator:

$$\mathbf{G}((valid \wedge \neg trap) \rightarrow \mathbf{X}(order = order_{prev} + 1))$$

$$\mathbf{G}(\neg valid \rightarrow \mathbf{X}(order = order_{prev}))$$

These properties are checked independently for both the DUV and the golden model, ensuring that each implementation maintains correct retirement ordering regardless of equivalence.

The PC alignment assertion verifies that program counter values respect the ISA's alignment requirements. For RV32I without the C extension, PCs must be 4-byte aligned; with the C extension, 2-byte alignment is permitted:

$$\mathbf{G}(valid \rightarrow (pc \bmod align = 0))$$

The alignment mask is computed dynamically based on the instruction encoding: compressed instructions (identified by `insn[1:0] != 2'b11`) allow halfword alignment.

To make formal proofs tractable, the generator includes assumptions that constrain the verification environment. These are not correctness properties but rather restrictions on the input space that reduce the state space explored by the model checker. Table 4.9 lists all generated assumptions with their corresponding LTL formulas. In addition to the notation defined for assertions, the assumptions table uses:

- *legal*: instruction encoding is valid for the configured ISA
- *op*: instruction opcode field (`insn[6:0]`)
- *seen_v*: at least one valid retirement has occurred
- *mem_seen*: a memory instruction has retired

- c_jump, c_mem : compressed jump or memory instruction
- is_ecall, is_ebreak : ECALL or EBREAK instruction
- is_pc_ctrl : instruction modifies control flow (branches, jumps)
- rst : reset signal
- N : configurable bounded window (cycles)

Table 4.9: Generated SVA assumptions with LTL semantics

Assumption	LTL Formula	Description
<i>Trap and Exception Handling</i>		
<code>no_trap</code>	$\mathbf{G}(\neg trap_d \wedge \neg trap_s)$	No traps from DUV or spec
<code>trap_illegal</code>	$\mathbf{G}((duv_v \wedge \neg legal) \rightarrow \mathbf{X}^{[0:1]} trap_d)$	Illegal insns must trap
<code>trap_never</code>	$\mathbf{G}(\neg trap_d)$	DUV trap always de-asserted
<i>Instruction Class Exclusions</i>		
<code>no_csr</code>	$\mathbf{G}(op \neq 1110011)$	Exclude CSR/privileged ops
<code>no_branch</code>	$\mathbf{G}(duv_v \rightarrow op \neq 1100011)$	Exclude branch instructions
<code>no_jump_branch</code>	$\mathbf{G}(duv_v \rightarrow op \notin \{B, J, JR\} \wedge \neg c_jump)$	Exclude all control flow
<code>no_mem</code>	$\mathbf{G}(duv_v \rightarrow \neg is_ld \wedge \neg is_st \wedge \neg c_mem)$	Exclude load/store insns
<code>no_fence</code>	$\mathbf{G}(duv_v \rightarrow op \neq 0001111)$	Exclude FENCE/FENCE.I
<code>no_ecall</code>	$\mathbf{G}(duv_v \rightarrow \neg is_ecall)$	Exclude ECALL instructions
<code>no_ebreak</code>	$\mathbf{G}(duv_v \rightarrow \neg is_ebreak)$	Exclude EBREAK instructions
<code>no_pc_control</code>	$\mathbf{G}(duv_v \rightarrow \neg is_pc_ctrl)$	Exclude PC-modifying insns
<i>Register Address Constraints (Ibex-specific)</i>		
<code>rs1_addr_matches_ibex</code>	$\mathbf{G}((duv_v \wedge use_rs1) \rightarrow rs1_a_d = insn[19:15])$	rs1 addr matches encoding
<code>rs2_addr_matches_ibex</code>	$\mathbf{G}((duv_v \wedge use_rs2) \rightarrow rs2_a_d = insn[24:20])$	rs2 addr matches encoding
<code>rs_addr_small_nc_ibex</code>	$\mathbf{G}((duv_v \wedge nc) \rightarrow rs1_a, rs2_a, rd_a \in \{0, 1, 2\})$	Constrain regs to x0-x2
<i>Memory Interface Constraints</i>		
<code>mem_signals_on_mem</code>	$\mathbf{G}((duv_v \wedge is_mem) \rightarrow rmask \neq 0 \vee wmask \neq 0)$	Mem masks set on mem ops

Continued on next page

Table 4.9 – continued from previous page

Assumption	LTL Formula	Description
eventually_mem_insn	$\mathbf{G}((seen_v \wedge \neg mem_seen) \rightarrow \mathbf{F}^{[1:N]} is_mem)$	Mem insn within N cycles
<i>Environment Constraints</i>		
reset_never	$\mathbf{G}(\neg rst)$	Reset stays deasserted

These assumptions can be selectively enabled or disabled via module parameters to expand verification coverage as confidence in the design grows. The default configuration enables only the assumptions necessary for tractable proofs while maximizing the instruction classes under verification.

The generator includes a helper function that determines whether an instruction encoding is legal for the configured ISA. Illegal instructions are expected to trap, and the framework does not compare architectural effects for trapped instructions.

Example 4.5.6: Instruction legality check function (excerpt)

```

1 function automatic bit rvavf_is_legal_insn(input logic [31:0] insn);
2   logic [6:0] opc;
3   logic [2:0] f3;
4   logic [6:0] f7;
5   begin
6     opc = insn[6:0];
7     f3  = insn[14:12];
8     f7  = insn[31:25];
9     rvavf_is_legal_insn = 1'b0;
10    unique case (opc)
11      7'b0110111: rvavf_is_legal_insn = 1'b1; // LUI
12      7'b0010111: rvavf_is_legal_insn = 1'b1; // AUIPC
13      7'b1101111: rvavf_is_legal_insn = 1'b1; // JAL
14      7'b1100111: rvavf_is_legal_insn = (f3 == 3'b000); // JALR
15      7'b0000011: rvavf_is_legal_insn = // LOAD
16        (f3 == 3'b000) || (f3 == 3'b001) || (f3 == 3'b010) ||
17        (f3 == 3'b100) || (f3 == 3'b101);
18      // ... additional cases for all RV32I instructions
19      default: rvavf_is_legal_insn = 1'b0;
20    endcase
21  end
22 endfunction

```

Each assertion is wrapped in a conditional generate block controlled by a parameter, as shown in the generated output above. This allows individual assertions to be disabled without regenerating the module, which is useful when debugging specific failures or when certain comparisons are not applicable to a particular DUV configuration.

4.5.3 Top-Level Harness

The `TopGenerator` class produces the `rvavf_top` module, which serves as the root of the formal verification hierarchy. This module instantiates three components: the RVGM golden model wrapped with an RVFI interface, the RVFIC-wrapped DUV, and the assertion module that compares their outputs.

The top-level module declares the RVFI signals as internal wires and connects them to both instantiated modules. The golden model signals are prefixed with `rvfic_spec_` while the DUV signals use the standard `rvfic_` prefix.

Example 4.5.7: Top-level harness structure (simplified)

```

1 module rvavf_top (
2     input logic clk,
3     input logic rst
4 );
5 // Golden model RVFI signals (specification)
6 logic rvfic_spec_valid;
7 logic [63:0] rvfic_spec_order;
8 logic [31:0] rvfic_spec_insn;
9 logic [XLEN-1:0] rvfic_spec_pc_rdata;
10 // ... additional spec signals
11 // DUV RVFI signals
12 logic rvfic_valid;
13 logic [63:0] rvfic_order;
14 logic [31:0] rvfic_insn;
15 logic [XLEN-1:0] rvfic_pc_rdata;
16 // ... additional DUV signals
17 // Golden model instance
18 RvgmRvficWrapper u_spec (
19     .clk(clk), .rst(rst),
20     .rvfic_valid(rvfic_spec_valid),
21     .rvfic_order(rvfic_spec_order),
22     // ...
23 );
24 // DUV instance
25 RvficWrapper u_duv (
26     .clk(clk), .rst(rst),
27     .rvfic_valid(rvfic_valid),
28     .rvfic_order(rvfic_order),
29     // ...
30 );
31 // Assertion module instance
32 rvavf_base_assertions u_assertions (
33     .clk(clk), .rst(rst),
34     .rvfic_spec_valid(rvfic_spec_valid),
35     .rvfic_valid(rvfic_valid),
36     // ...
37 );
38 endmodule

```

The assertion module receives both signal sets as inputs and evaluates all enabled assertions on every clock cycle. The formal tool explores the state space by driving symbolic values into the shared inputs (clock, reset, and instruction fetch interface) and checking whether any assertion can be violated.

The generator reads the active CSR signals from the RVFIC map file and creates matching port connections for both modules. This ensures that the same CSR fields are compared regardless of which CSRs the DUV actually implements. When a CSR signal is present in the map but not in

the DUV's interface, the generator ties it to zero and disables the corresponding assertions.

The top-level module passes assertion enable parameters to the assertion module instance. These parameters are read from environment variables at generation time and converted to SystemVerilog parameter assignments.

Example 4.5.8: Assertion enable parameter propagation

```

1 rvavf_base_assertions #(
2   .XLEN(XLEN),
3   .MAX_LAG(32),
4   .RVAVF_ENABLE_ASSERT_MATCH_VALID_IBEX(1'b1),
5   .RVAVF_ENABLE_ASSERT_MATCH_INSN_IBEX(1'b1),
6   .RVAVF_ENABLE_ASSERT_MATCH_GPR_RD_ADDR_IBEX(1'b1),
7   .RVAVF_ENABLE_ASSERT_MATCH_GPR_RD_WDATA_BOOT_IBEX(1'b1),
8   .RVAVF_ENABLE_ASSUME_NO_TRAP(1'b1),
9   .RVAVF_ENABLE_ASSUME_NO_FENCE(1'b1)
10  // ... additional parameters
11 ) u_assertions (
12   .clk(clk),
13   .rst(rst),
14   // ... signal connections
15 );

```

This allows the user to disable specific assertions from the Makefile without modifying generated code.

4.5.4 Formal Verification Flow

The formal verification flow is orchestrated by the `make rvavf.formal` target, which executes a sequence of generation steps and produces a workspace ready for JasperGold. The flow is fully automated: given a configuration, it generates all required artifacts without manual intervention.

The formal flow consists of seven sequential stages:

1. **Clean.** Remove prior outputs to ensure a fresh build.
2. **Generate assertions.** Invoke the `AssertionGenerator` (Section 4.5.2).
3. **Generate top-level harness.** Invoke the `TopGenerator` (Section 4.5.3).
4. **Generate golden model.** Run the RVGM generator (Section 4.4).
5. **Generate DUV wrapper.** Run the RVFIC generator (Section 4.3).
6. **Generate TCL script.** Invoke the `TclGenerator`.
7. **Run JasperGold.** Execute the formal tool with the generated script and collect results.

The `TclGenerator` collects all HDL sources into a structured workspace. Workspace management, file staging, and TCL script generation for the formal tool are handled by CHIPS (Chisel Hardware

Integration and Verification Services), an external library that provides backend integration for commercial formal verification tools. The staging directory contains:

- `verilog/` — all SystemVerilog files (RVGM, RVFIC, DUV, assertions)
- `CHIPS_FPV.tcl` — entry point for JasperGold
- `cex_traces/` — counterexample dump location (if enabled)

The generated TCL script configures the JasperGold environment, reads all staged files, sets the clock and reset signals, and issues the proof command. Due to licensing restrictions on Cadence tools, the specific TCL commands cannot be disclosed; the example below shows a simplified structure.

Example 4.5.9: JasperGold TCL script structure (simplified)

```
1 # Clock and reset configuration
2 clock clk
3 reset rst
4 # Read design files
5 analyze -sv rvavf_top.sv
6 analyze -sv RvgmRvficWrapper.sv
7 analyze -sv RvficWrapper.sv
8 analyze -sv assertions_rvgm_vs_ibex.sv
9 # ... additional source files
10 # Elaborate top module
11 elaborate -top rvavf_top
12 # Configure proof engine
13 set_prove_time_limit 3600
14 set_max_trace_length 100
15 # Run bounded model checking
16 prove -all
```

JasperGold reports each assertion as proven, falsified, or undetermined. A proven assertion holds for all reachable states within the bounded depth. A falsified assertion includes a counterexample trace showing the input sequence that leads to the violation. An undetermined assertion could not be resolved within the time or memory limits.

This chapter presents the formal verification results obtained by applying the RVAVF framework to four open-source RISC-V processor cores. The evaluation covers the experimental setup, verification results, analysis of counterexamples, and a discussion of findings.

5.1 Experimental Setup

The evaluation targets four open-source RISC-V processor cores with varying complexity and ISA support. Each core was wrapped using RVFIC and verified against RVGM through the RVAVF formal flow. JasperGold was used as the formal verification engine.

5.1.1 Target Cores

The following cores were selected for evaluation.

- **PicoRV32**: A size-optimized RV32IM core designed for FPGA implementations. It supports the base integer instruction set and the multiply/divide extension.
- **NERV**: A minimal RV32I core that implements only the base integer instruction set. Its simple microarchitecture makes it suitable for baseline verification.
- **Ibex**: A production-quality RV32IMC core developed by lowRISC. It supports the base integer, multiply/divide, and compressed instruction extensions. The Zicsr extension is also enabled for CSR access.
- **Hazard3**: An RV32IM core with a three-stage pipeline. It supports the base integer and multiply/divide extensions.

An attempt was made to include the CVA6¹ core in the evaluation. However, CVA6 implements RVFI ports in a non-standard manner that does not align with the expected signal semantics. The verification assertions could not produce meaningful results, so CVA6 was excluded from the final evaluation.

¹<https://github.com/openhwgroup/cva6>

5.1.2 Verification Commands

Each core was verified using the `make rvavf.formal` target with core-specific configuration presets. The following commands were used for reproducibility.

- **Ibex:** `RVAVF_CORE=ibex_rv32czicr_mem RVAVF_CEX_TRACES=1
RVAVF_JG_NON_RESETTABLE_REGS=0 RVFIC_BOOT_ADDR="32'h00000000"
RVGM_PC_START=0x80 make rvavf.formal`
- **NERV:** `RVAVF_CORE=nerv_rv32i_mem RVAVF_CEX_TRACES=1
RVAVF_JG_NON_RESETTABLE_REGS=0 make rvavf.formal`
- **PicoRV32:** `RVAVF_CORE=pico_rv32im_mem RVAVF_CEX_TRACES=1
RVAVF_JG_NON_RESETTABLE_REGS=0 make rvavf.formal`
- **Hazard3:** `RVAVF_CORE=hazard3_rv32im_mem RVAVF_CEX_TRACES=1
RVAVF_JG_NON_RESETTABLE_REGS=0 make rvavf.formal`

The environment variables control the verification setup as follows.

- **RVAVF_CORE:** Selects a predefined core configuration preset that sets the ISA extensions, memory interface options, and RVFIC map file for the target processor.
- **RVAVF_CEX_TRACES:** When set to 1, JasperGold generates VCD waveform files for any counterexamples found during verification.
- **RVAVF_JG_NON_RESETTABLE_REGS:** When set to 0, injects a TCL command that instructs JasperGold to initialize all registers to zero at the start of verification. This ensures the processor and golden model begin from the same register state.
- **RVFIC_BOOT_ADDR:** Sets the boot address for the RVFIC wrapper. For Ibex, this is set to 0x00000000.
- **RVGM_PC_START:** Sets the initial program counter value for the golden model. Ibex begins instruction fetch at address 0x80, so RVGM must be configured to start at the same address for the program counter assertions to match.

The Ibex command requires additional address configuration because Ibex starts fetching instructions from address 0x80 rather than 0x00. The RVFIC boot address and RVGM program counter start must be aligned so that both the processor and golden model agree on the initial PC value when the first instruction retires.

5.1.3 State Space Reduction

Formal verification of processor cores faces the challenge of state space explosion. Without constraints, JasperGold would need to explore an enormous number of possible execution states, and verification runs would not converge. Two techniques were employed to make verification tractable.

The first technique limits temporal depth. Instead of verifying all retired instructions, the assertions check correctness only at the first and second instruction retirements. This is reflected in the assertion naming convention with `_boot` suffixes for the first retired instruction and `_second`

suffixes for the second. Extending verification beyond two instructions causes the state space to grow beyond what JasperGold can handle in reasonable time.

This approach is valid for the scope of instructions verified in this work. Instructions like `ADD` or `SUB` do not accumulate hidden state that would change their behavior over time. If the `ADD` instruction produces the correct result for the first and second retirements, it will produce the correct result for the third retirement and beyond. The instruction reads its source registers, computes the sum, and writes the destination register in the same way regardless of how many instructions came before. However, this assumption may not hold for more complex RISC-V extensions. Instructions that depend on internal microarchitectural state, such as floating-point status flags or vector register grouping, might behave differently after several retirements. Verifying such extensions would require deeper temporal bounds or alternative techniques.

The second technique constrains the register address space. The `assume_rs_addr_small_nc` assumption restricts instructions to use only registers `x0`, `x1`, and `x2` instead of the full 32-register file. This reduces the number of register combinations that must be explored while still exercising the core register read and write logic.

These constraints reduce verification completeness but enable the runs to converge. The underlying hardware logic is identical regardless of which registers are used, so proving correctness for `x0`, `x1`, and `x2` provides confidence in the overall implementation.

The verification runs on a virtual machine with limited memory. Large state spaces cause JasperGold to run out of memory and crash before completing. The constraints on temporal depth and register addresses keep the state space small enough for verification to finish within available memory.

5.2 Verification Results

Table 5.1 lists the assumptions applied during formal verification for each processor. These assumptions constrain the instruction sequences that JasperGold explores, focusing verification on supported operations.

The `assume_no_trap` assumption excludes instructions that would cause traps. The `assume_no_fence` assumption excludes fence instructions. The `assume_rs_addr_small_nc` assumption restricts register addresses to `x0`, `x1`, and `x2` as discussed in Section 5.1.3. The `assume_no_ecall` and `assume_no_ebreak` assumptions exclude environment call and breakpoint instructions for cores where these would cause verification issues.

Table 5.1: Formal Verification Assumptions per Processor

Assumption	PicoRV32	NERV	Ibex	Hazard3
<code>assume_no_trap</code>	✓	✓	✓	✓
<code>assume_no_fence</code>	✓	✓	✓	✓
<code>assume_rs_addr_small_nc</code>	✓	✓	✓	✓
<code>assume_no_ecall</code>	—	✓	—	✓
<code>assume_no_ebreak</code>	—	✓	—	✓

Table 5.2 presents the formal verification results for each assertion across all four processor cores. The possible outcomes are as follows.

- **Proven:** The assertion was formally proven to hold for all possible execution traces within the verification bounds.
- **CEX:** A counterexample was found, indicating a mismatch between the processor and the golden model.
- **Unreachable:** The assertion's precondition could not be satisfied, meaning the scenario under test cannot occur in the current verification setup.
- **—:** The assertion does not apply to the processor configuration. This occurs for compressed instruction assertions when the core does not support the C extension.

Table 5.2: Formal Verification Results: RISC-V Processors vs. Assertions

Assertion	PicoRV32	NERV	Ibex	Hazard3
<i>Control Flow Signals</i>				
match_valid	Proven	Proven	Proven	Proven
match_order	CEX	Proven	Proven	CEX
match_insn	Proven	Proven	Proven	Proven
duv_order_increments	Proven	Proven	Proven	Proven
<i>Program Counter</i>				
match_pc_rdata_boot	Proven	Proven	Proven	Proven
match_pc_rdata_second	Proven	Proven	Proven	Proven
match_pc_wdata_boot	Proven	Proven	Proven	Proven
match_pc_wdata_second	Proven	Proven	Proven	CEX
match_pc_wdata_no_jump_boot	Proven	Proven	Proven	Proven
match_pc_wdata_no_jump_second	Proven	Proven	Proven	CEX
<i>Register File - Destination</i>				
match_gpr_rd_addr	Proven	Proven	Proven	Proven
match_gpr_rd_wdata_boot	Proven	Proven	Proven	CEX
match_gpr_rd_wdata_second	Proven	Proven	Proven	CEX
<i>Register File - Source (Non-Compressed)</i>				
match_gpr_rs1_addr_nc	Proven	Proven	Proven	Proven
match_gpr_rs1_rdata_nc_boot	Unreachable	Proven	Proven	Proven
match_gpr_rs1_rdata_nc_second	Proven	Proven	Proven	CEX
match_gpr_rs2_rdata_nc_boot	Unreachable	Proven	Proven	Proven
match_gpr_rs2_rdata_nc_second	Unreachable	Proven	Proven	CEX
<i>Register File - Source (Compressed)</i>				
match_gpr_rs1_addr_c	—	—	Proven	—
match_gpr_rs1_rdata_c_boot	—	—	Proven	—
match_gpr_rs1_rdata_c_second	—	—	Proven	—
match_gpr_rs2_addr_nc	—	—	Proven	—
match_gpr_rs2_addr_c	—	—	Proven	—
match_gpr_rs2_rdata_c_boot	—	—	Proven	—
match_gpr_rs2_rdata_c_second	—	—	Proven	—
<i>Memory Interface</i>				
match_mem_addr	Proven	Proven	Unreachable	CEX
match_mem_masks	Proven	Proven	Unreachable	CEX
match_mem_rdata	Unreachable	Proven	Proven	Proven
match_mem_wdata	Proven	Proven	Unreachable	CEX

NERV achieved the cleanest results with all 21 applicable assertions proven and no counterexamples. Ibex had 25 assertions proven with 3 unreachable due to memory interface limitations discussed in Section 5.3.4. PicoRV32 had 16 assertions proven, 1 counterexample, and 4 unreachable assertions. Hazard3 had the most counterexamples with 10 out of 21 applicable assertions failing.

5.3 Analysis of Results

The counterexamples and unreachable assertions reveal important insights about RVFI implementations across different cores. This section analyzes the root causes of verification failures and limitations.

5.3.1 Hazard3 RVFI Implementation

All 10 counterexamples found for Hazard3 stem from a non-standard RVFI implementation. The `rvfi_insn` signal in Hazard3 does not show the currently retired instruction. Instead, it continuously outputs the instruction stream regardless of the `rvfi_valid` signal state.

When `rvfi_valid` is asserted to indicate instruction retirement, the `rvfi_insn` value may not correspond to the instruction actually retiring at that cycle. The RVFI specification expects `rvfi_insn` to hold the retired instruction word when `rvfi_valid` is high. Hazard3's deviation from this expectation causes all instruction-dependent assertions to fail.

Figure 5.1 shows a counterexample waveform demonstrating this behavior. The mismatch is not a bug in Hazard3's execution logic but rather a difference in how the developers chose to implement RVFI signaling.

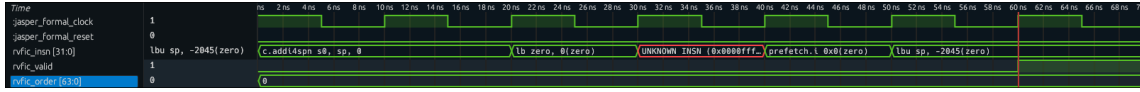


Figure 5.1: Counterexample waveform showing Hazard3 `rvfi_insn` behavior

5.3.2 PicoRV32 Order Counter Timing

The single counterexample for PicoRV32 occurs in the `match_order` assertion. The RVFI specification expects the order counter to increment when `rvfi_valid` becomes high, indicating that the new order value should be available at the same cycle as the retirement signal.

PicoRV32 increments the order counter after `rvfi_valid` returns to zero. This means the updated order value appears one cycle after the retirement, which causes a mismatch with the golden model that updates its order counter synchronously with the valid signal.

Figure 5.2 shows a counterexample waveform illustrating this timing difference. Like Hazard3, this is not an execution bug but a deviation from expected RVFI timing semantics.

5.3.3 PicoRV32 Register File Initialization

The `RVAVF_JG_NON_RESETTABLE_REGS=0` environment variable injects a TCL command that instructs JasperGold to initialize all registers to zero, ensuring both the processor and golden model start

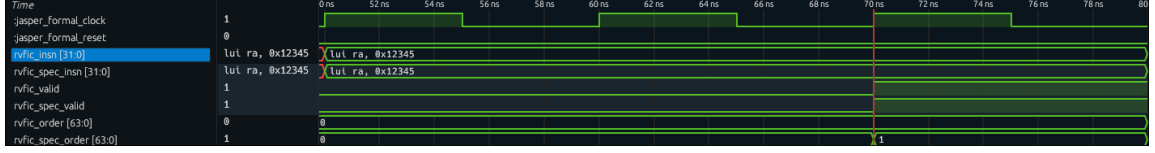


Figure 5.2: Counterexample waveform showing PicoRV32 order counter timing

from the same state. However, this mechanism does not work for PicoRV32. The register file retains arbitrary symbolic values at verification start, while RVGM’s registers are initialized to zero.

Some cores require specific parameters to be passed during elaboration for certain features to function correctly. Due to time constraints, PicoRV32 was not investigated further to determine whether such a parameter exists or why the standard register initialization fails. The consequence is that the initial register states never match between PicoRV32 and RVGM, making four assertions unreachable.

To validate that PicoRV32’s register file logic is correct despite this limitation, verification was conducted with a minimal program constraint. The constraint requires the first instruction to be a LUI that loads a value into x1, followed by an ADDI that uses x1 as a source register. By the second instruction, x1 has a known value written by the first instruction, allowing the `match_gpr_rs1_rdata_nc_second` assertion to be proven. This confirms that PicoRV32’s register reads are correct once the initial state mismatch is overcome.

5.3.4 Memory Interface Limitations

The RVAVF formal harness does not include an external memory model. Without a model driving the data-side handshake signals, memory transactions cannot complete. This affects memory-related assertions differently across cores.

For Ibex, the load-store unit requires a memory response to complete any memory transaction. Since no response arrives in the formal environment, no load or store instruction ever retires. The RVFIC memory signals remain inactive, and the memory assertions become unreachable. The assertions `match_mem_addr`, `match_mem_masks`, and `match_mem_wdata` are all unreachable for Ibex.

For PicoRV32, only `match_mem_rdata` is unreachable. Store instructions can still retire and activate the write-side memory signals, but load instructions require a valid memory response to provide meaningful read data. Without a memory model, loads do not retire with valid `rvfi_mem_rdata` values.

NERV has no unreachable memory assertions because its microarchitecture allows load and store instructions to retire without depending on an external memory handshake.

Hazard3 shows counterexamples rather than unreachable assertions for memory signals. This is because the `rvfi_insn` issue discussed in Section 5.3.1 causes mismatches before the memory reachability question becomes relevant.

5.4 RVFIC Signal Contract

The evaluation results reveal that verification failures often stem from differences in how cores implement RVFI signals rather than actual bugs in processor logic. Hazard3’s instruction signal

timing and PicoRV32’s order counter behavior are both reasonable implementation choices, yet they produce counterexamples when compared against a reference model with different timing expectations. The absence of a memory model causes assertions to become unreachable for cores that require external handshakes to retire memory instructions. These findings lead to the formulation of a signal contract that defines the requirements for RVFI implementations to be compatible with formal verification frameworks.

The contract is built on two fundamental rules.

Definition 5.4.1 (Retirement Cycle Rule). *RVFIC signals are meaningful only on the cycle when a new instruction retires.*

$$\begin{aligned} \text{retire} &\equiv (\text{rvfic_valid} \wedge (\text{rvfic_insn} \neq \text{insn_prev})) \\ \mathbf{G}(\text{retire} \rightarrow \text{rvfic_holds}) \end{aligned}$$

where *rvfic_holds* denotes that all RVFIC signals satisfy their requirements as defined in Table 5.3.

Real processor cores connect to memory through complex protocols such as AXI4 or TileLink with multi-cycle handshakes and acknowledgment signals. The RVAVF framework verifies ISA-level correctness, not memory subsystem behavior. A core that waits indefinitely for an external memory acknowledgment will never assert `rvfic_valid` for memory instructions, making those assertions unreachable. The memory handshake rule addresses this by requiring cores to provide a verification mode where memory operations complete immediately.

Definition 5.4.2 (Memory Handshake Rule). *Memory instructions must retire without waiting for external acknowledgment.*

$$\mathbf{G}(\text{is_mem_op} \rightarrow \mathbf{F}(\text{rvfic_valid}))$$

where *is_mem_op* is true when the current instruction is a load or store.

Table 5.3 defines the contract for each RVFIC signal. Each signal must satisfy its requirement when `rvfic_valid` is asserted.

Cores that follow this contract can be verified with RVAVF without encountering the counterexamples and unreachable assertions observed in the evaluation. The contract represents a concrete outcome of this thesis, translating verification findings into actionable requirements for RVFI implementations.

Table 5.3: RVFIC signal contract at retirement time.

Signal	Contract requirement (when <code>rvfic_valid</code> is high)
<code>rvfic_valid</code>	Asserts for exactly the cycle in which the instruction’s architectural effects are committed (retirement).
<code>rvfic_order</code>	Monotonic retirement counter; increments on each non-trapping retirement and remains stable otherwise.
<code>rvfic_trap</code>	High iff the retired instruction raises a trap (illegal instruction or exception).
<code>rvfic_insn</code>	Retired instruction word corresponding to the committed architectural state.
<code>rvfic_pc_rdata</code>	Program counter value of the retired instruction (before execution).
<code>rvfic_pc_wdata</code>	Next architectural program counter after retirement.
<code>rvfic_rs1_addr</code> , <code>rvfic_rs2_addr</code>	Source register indices; set to zero when the instruction does not read the corresponding source.
<code>rvfic_rs1_rdata</code> , <code>rvfic_rs2_rdata</code>	Source register values actually read; meaningful only when the instruction reads that source.
<code>rvfic_rd_addr</code>	Destination register index; set to zero when no register is written.
<code>rvfic_rd_wdata</code>	Value written back to the destination register; meaningful only when <code>rvfic_rd_addr</code> is nonzero.
<code>rvfic_mem_addr</code>	Effective memory address for loads/stores; zero when no memory access occurs.
<code>rvfic_mem_rmask</code> , <code>rvfic_mem_wmask</code>	Byte-enable masks for memory reads/writes; zero when no memory access occurs.
<code>rvfic_mem_rdata</code>	Load data for memory reads; must be provided in the retirement cycle for loads.
<code>rvfic_mem_wdata</code>	Store data for memory writes; valid in the retirement cycle for stores.

5.5 Discussion

The evaluation results demonstrate both the capabilities and limitations of the RVAVF formal verification framework. A significant finding is the variation in RVFI implementations across cores. The RVFI specification defines signal names and intended semantics, but leaves room for interpretation in timing and edge cases. Hazard3’s continuous instruction stream on `rvfi_insn` and PicoRV32’s delayed order counter increment are both valid interpretations from the developers’ perspectives, yet they cause verification failures when compared against a reference that follows stricter semantics.

This shows the need for more precise RVFI timing specifications. A formal verification framework can only produce correct results when both the design under test and the golden model agree on signal timing. The counterexamples found in this evaluation are not processor bugs but RVFI implementation differences.

Despite the state space reduction techniques, the framework successfully proves correctness for the verified scenarios. NERV’s complete pass with 21 proven assertions demonstrates that a well-aligned RVFI implementation can be fully verified. Ibex’s 25 proven assertions show that even

complex cores with compressed instruction support can be verified when RVFI is implemented according to expected semantics.

Verifying only the first two retired instructions limits the scope of the analysis. Longer instruction sequences that might expose bugs through accumulated state or specific instruction orderings are not covered. The framework can extend to verify three, four, or more retirements, but each additional instruction multiplies the state space that JasperGold must explore. This increases both memory consumption and analysis runtime. On the resource-constrained virtual machine used for this evaluation, extending beyond two retirements caused verification runs to exhaust available memory before completing.

The tradeoff between verification depth and resource consumption can be managed through additional assumptions. Constraining the instruction encoding space, limiting memory access patterns, or restricting control flow reduces the number of states the solver must consider. With sufficiently strong assumptions, verification of deeper instruction sequences becomes feasible, though at the cost of reduced coverage.

For the instructions verified in this work, the two-instruction limit provides meaningful coverage. Processor logic for these instructions is combinational and does not accumulate hidden state across retirements. An `ADD` instruction executes identically whether it is the second or the hundredth instruction in a sequence. Proving correctness for two retirements therefore provides confidence that subsequent executions will also be correct.

The evaluation shows that RVAVF is applicable to cores with RVFI implementations. NERV and Ibex produced meaningful verification results with proven assertions and explainable unreachable cases. Cores with differing RVFI implementations like Hazard3 and CVA6 require either modification to their RVFI logic or adaptation of the verification framework to match their signal semantics.

The memory interface limitation affects all cores that require external memory handshakes. Adding a formal memory model to the harness would enable memory assertion verification but would also increase the state space. The dummy memory interface represented in the RVFIC Contract is a good solution for this.

Conclusion and Future Work | 6

6.1 Conclusion

This thesis addresses the scalability gap between generator-based processor design and verification. When a single generator can emit many architectural variants, a hand-built verification environment no longer scales. The key idea in this work is to treat verification itself as a generator problem. Given the same configuration that defines the hardware, the framework generates the reference model, the observation interface, and the formal harness automatically.

The solution is realized as three cooperating generators. RVFIC produces a uniform RVFI-style interface by generating bind probes and wrappers from a signal map, enabling non-intrusive observation of any core without modifying its source code. RVGM provides a synthesizable golden model that computes the architectural effects of each retired instruction for the selected ISA configuration. RVAVF integrates both sides and emits SystemVerilog assertions that compare retirement transactions signal-by-signal. Together, these components create a complete formal environment from a single configuration and reduce verification to an automated, repeatable flow. A further contribution is the RVFIC signal contract introduced in Chapter 5, which precisely defines the retirement-cycle requirements that cores must satisfy for consistent verification results.

The evaluation demonstrates that the approach is effective when RVFI semantics are aligned. NERV and Ibex achieve broad proof coverage, while counterexamples in Hazard3 and PicoRV32 reveal mismatches in signal timing rather than functional bugs. These findings confirm that the framework successfully distinguishes interface inconsistencies from actual processor errors.

The framework has limitations that bound the scope of its applicability. Verification depth is constrained to the first two retired instructions due to state space explosion in bounded model checking. The CSR semantics in the golden model remain incomplete and do not yet match all implementation-specific behaviors. Additionally, integrating a new core still requires manually constructing a signal map that describes how internal signals correspond to RVFI fields.

Returning to the central question posed in the introduction, this thesis demonstrates that a generator-based framework leveraging a common formal interface can indeed enable automated and scalable formal verification of RISC-V core implementations. The approach succeeds when cores adhere to consistent interface semantics, and it provides actionable diagnostic information when

they do not. By generating verification environments from the same configuration that defines the hardware, the framework makes formal methods practical for agile processor development and opens a path toward continuous ISA compliance checking as part of the design cycle.

6.2 Future Work

Several extensions can strengthen and broaden the framework.

- **Formalize the RVFIC contract:** Publish the contract as a precise, testable specification of retirement-cycle requirements so cores can implement RVFIC consistently and verification setups share a common semantic baseline.
- **Expand RVGM ISA coverage:** Extend the golden model to additional ISA extensions such as A (atomics), F/D (floating point), vector, and privileged modes, while keeping the model modular and tractable for formal tools.
- **Complete CSR semantics:** The current CSR handling is known to be incomplete. A more faithful CSR model would improve agreement with real cores, particularly for machine-mode counters and status registers.
- **Tool portability:** Generate harnesses and scripts for other formal backends (e.g., SymbiYosys¹ or OneSpin²) to reduce reliance on a single tool and make the flow more accessible.
- **Increased automation:** Assist users in building signal maps by leveraging debug metadata, naming heuristics, or semi-automatic inference, reducing manual effort when integrating new cores. AI agents could analyze open-source core repositories and automatically construct RVFIC wrappers by identifying signal patterns and matching them to RVFI semantics.

¹<https://github.com/YosysHQ/sby>

²<https://www.siemens.com/en-us/products/ic/questa-one/formal-verification/>

Bibliography

- [1] H. D. Foster, “Trends in functional verification: A 2014 industry study,” *Proceedings of the 52nd Annual Design Automation Conference*, pp. 1–6, 2015.
- [2] *The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 20191214-draft*, RISC-V Foundation, Dec. 2019, draft version.
- [3] S. . YosysHQ, “Risc-v formal verification framework and rvfi specification,” <https://github.com/SymbioticEDA/riscv-formal>, 2025, accessed: 2025-12-11.
- [4] A. Izraelevitz, J. Koenig, P. Li, R. Lin, A. Wang, A. Magyar, D. Kim, C. Schmidt, C. Markley, J. Lawson, and J. Bachrach, “Reusability is firrtl ground: Hardware construction languages, compiler frameworks, and transformations,” in *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2017, pp. 209–216.
- [5] A. Pnueli, “The temporal logic of programs,” in *18th Annual Symposium on Foundations of Computer Science (sfcs 1977)*, 1977, pp. 46–57.
- [6] E. Cerny, S. Dudani, J. Havlicek, and D. Korchemny, *SVA: The Power of Assertions in SystemVerilog*. Springer Cham, 2015.
- [7] A. B. Mehta, *System Verilog Assertions and Functional Coverage: Guide to Language, Methodology and Applications*. Springer Cham, 2020.
- [8] I. S. Association, “Ieee standard for systemverilog—unified hardware design, specification, and verification language,” *IEEE Std 1800-2017 (Revision of IEEE Std 1800-2012)*, pp. 1–1315, 2018.
- [9] J. R. Burch and D. L. Dill, “Automatic verification of pipelined microprocessor control,” in *Computer Aided Verification*, D. L. Dill, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1994, pp. 68–80.
- [10] D. Gao and T. Melham, “End-to-end formal verification of a RISC-V processor extended with capability pointers,” in *Formal Methods in Computer Aided Design, FMCAD 2021, New Haven, CT, USA, October 19-22, 2021*. IEEE, 2021, pp. 24–33. [Online]. Available: https://doi.org/10.34727/2021/isbn.978-3-85448-046-4_10
- [11] L.-E. Ploix, A. Armstrong, T. Melham, R. Lin, H. Wang, and A. Courtney, “Comprehensive formal verification of observational correctness for the cheriot-ibex processor,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.04738>

- [12] S. Davidmann, L. Moore, R. Ho, T. Liu, D. Letcher, and A. Sutton, “Rolling the dice with random instructions is the safe bet on risc-v verification,” *DVCON 2020, Tech. Rep.*, 2020.
- [13] N. Bruns, V. Herdt, and R. Drechsler, “Processor verification using symbolic execution: A risc-v case-study,” in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2023, pp. 1–6.
- [14] M. Xiang, Y. Li, and Y. Zhao, “Rvfc: Risc-v formal in chisel,” in *2023 International Symposium of Electronics Design Automation (ISED)*, 2023, pp. 162–167.

Appendix A

A.1 Source Code Repositories

The source code for the framework components is available in the following repositories:

- RVAVF: <https://github.com/onuralt/RVAVF>
- RVGM: <https://github.com/onuralt/RVGM>
- RVFIC: <https://github.com/onuralt/RVFIC>

A.2 PicoRV32 JSON Map

Example A.2.1: rvfic_map_picorv32.json

```
1 {
2   "duvModule": "picorv32",
3   "clock": "clk",
4   "reset": { "name": "resetn", "activeLow": true },
5   "parameters": {
6     "ENABLE_COUNTERS": 1,
7     "ENABLE_TRACE": 1,
8     "ENABLE_REGS_DUALPORT": 1,
9     "RISCV_FORMAL": 1,
10    "ENABLE_IRQ": 0,
11    "PROGADDR_IRQ": "32'h00000020",
12    "CATCH_ILLINSN": 0,
13    "CATCH_MISALIGN": 0
14  },
15   "signals": {
16     "rvfic_valid": { "path": "rvfi_valid", "width": 1 },
17     "rvfic_order": { "path": "rvfi_order", "width": 64 },
```

```

18   "rvfic_trap": { "path": "rvfi_trap", "width": 1 },
19   "rvfic_insn": { "path": "rvfi_insn", "width": 32 },
20   "rvfic_pc_rdata": { "path": "rvfi_pc_rdata", "width": 32 },
21   "rvfic_pc_wdata": { "path": "rvfi_pc_wdata", "width": 32 },
22   "rvfic_rs1_addr": { "path": "rvfi_rs1_addr", "width": 5 },
23   "rvfic_rs2_addr": { "path": "rvfi_rs2_addr", "width": 5 },
24   "rvfic_rs1_rdata": { "path": "rvfi_rs1_rdata", "width": 32 },
25   "rvfic_rs2_rdata": { "path": "rvfi_rs2_rdata", "width": 32 },
26   "rvfic_rd_addr": { "path": "rvfi_rd_addr", "width": 5 },
27   "rvfic_rd_wdata": { "path": "rvfi_rd_wdata", "width": 32 },
28   "mem_valid": { "name": "mem_valid", "dir": "output", "width": 1, "type": "port"
29   },
30   "mem_instr": { "name": "mem_instr", "dir": "output", "width": 1, "type": "port"
31   },
32   "mem_ready": { "name": "mem_ready", "dir": "input", "width": 1, "type": "port" }
33   ,
34   "mem_addr": { "name": "mem_addr", "dir": "output", "width": 32, "type": "port" }
35   ,
36   "mem_wdata": { "name": "mem_wdata", "dir": "output", "width": 32, "type": "port"
37   },
38   "mem_wstrb": { "name": "mem_wstrb", "dir": "output", "width": 4, "type": "port"
39   },
40   "mem_rdata": { "name": "mem_rdata", "dir": "input", "width": 32, "type": "port"
41   }
42 }

```

A.3 PicoRV32 Internal JSON Map

Example A.3.1: rvfic_map_picorv32_internal.json

```

1 {
2   "duvModule": "picorv32",
3   "clock": "clk",
4   "reset": { "name": "resetn", "activeLow": true },
5   "enableMemorySignals": true,
6   "parameters": {
7     "ENABLE_COUNTERS": 1,
8     "ENABLE_TRACE": 1,
9     "ENABLE_REGS_DUALPORT": 1,
10    "ENABLE_IRQ": 0,
11    "PROGADDR_IRQ": "32'h00000020",
12    "CATCH_ILLINSN": 1,
13    "CATCH_MISALIGN": 0
14  },
15   "signals": {

```

```

16  "rvfic_valid": { "path": "resethn && dbg_valid_insn && ((launch_next_insn && !
    dbg_next) || trap_pulse)", "width": 1 },
17  "rvfic_order": { "path": "COUNTER", "width": 64 },
18  "rvfic_trap": { "path": "trap", "width": 1 },
19  "rvfic_insn": { "path": "dbg_insn_opcode", "width": 32 },
20  "rvfic_pc_rdata": { "path": "dbg_insn_addr", "width": 32 },
21  "rvfic_pc_wdata": { "path": "dbg_irq_call ? dbg_irq_ret : dbg_insn_addr", "width":
    32 },
22  "rvfic_rs1_addr": { "path": "dbg_rs1val_valid ? dbg_insn_rs1 : 5'b00000", "width":
    5 },
23  "rvfic_rs2_addr": { "path": "dbg_rs2val_valid ? dbg_insn_rs2 : 5'b00000", "width":
    5 },
24  "rvfic_rs1_rdata": { "path": "dbg_rs1val_valid ? dbg_rs1val : 32'b0", "width":
    32 },
25  "rvfic_rs2_rdata": { "path": "dbg_rs2val_valid ? dbg_rs2val : 32'b0", "width":
    32 },
26  "rvfic_rd_addr": { "path": "cpuregs_write && !irq_state ? latched_rd : 5'b0", "width":
    5 },
27  "rvfic_rd_wdata": { "path": "cpuregs_write && !irq_state && latched_rd ?
    cpuregs_wdata : 32'b0", "width": 32 },
28  "rvfic_mem_addr": { "path": "(!dbg_irq_call && !dbg_mem_instr && dbg_mem_valid
    && dbg_mem_ready) ? dbg_mem_addr : 32'b0", "width": 32 },
29  "rvfic_mem_rmask": { "path": "(!dbg_irq_call && !dbg_mem_instr && dbg_mem_valid
    && dbg_mem_ready) ? (dbg_mem_wstrb ? 4'b0000 : 4'b1111) : 4'b0000", "width": 4 },
30  "rvfic_mem_wmask": { "path": "(!dbg_irq_call && !dbg_mem_instr && dbg_mem_valid
    && dbg_mem_ready) ? dbg_mem_wstrb : 4'b0000", "width": 4 },
31  "rvfic_mem_rdata": { "path": "(!dbg_irq_call && !dbg_mem_instr && dbg_mem_valid
    && dbg_mem_ready) ? dbg_mem_rdata : 32'b0", "width": 32 },
32  "rvfic_mem_wdata": { "path": "(!dbg_irq_call && !dbg_mem_instr && dbg_mem_valid
    && dbg_mem_ready) ? dbg_mem_wdata : 32'b0", "width": 32 },
33  "mem_valid": { "name": "mem_valid", "dir": "output", "width": 1, "type": "port"
    },
34  "mem_instr": { "name": "mem_instr", "dir": "output", "width": 1, "type": "port"
    },
35  "mem_ready": { "name": "mem_ready", "dir": "input", "width": 1, "type": "port"
    },
36  "mem_addr": { "name": "mem_addr", "dir": "output", "width": 32, "type": "port"
    },
37  "mem_wdata": { "name": "mem_wdata", "dir": "output", "width": 32, "type": "port"
    },
38  "mem_wstrb": { "name": "mem_wstrb", "dir": "output", "width": 4, "type": "port"
    },
39  "mem_rdata": { "name": "mem_rdata", "dir": "input", "width": 32, "type": "port"
    }
40  }
41  }

```